Image Stenography

A

PROJECT-III

SUBMITTED IN THE PARTIAL FULFILLMENT
FOR THE AWARD OF THE
DEGREE OF
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING

Submitted by
Shubhanshu (8519155)

Under the guidance of
Dr. Vishal Garg (A.P.)

Department of Computer Science and Engineering
Jai Parkash Mukand Lal Innovative Engineering & Technology Institute, (JMIETI) Radaur
(Kurukshetra University, Kurukshetra)
(2019-2023)

Jai Parkash Mukand Lal Innovative Engineering & Technology Institute

(JMIETI) Radaur

Approved by AICTE & Affiliated to Kurukshetra University, Kurukshetra

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that SHUBHANSHU SHUKLA, Roll No 8519155, has completed his Project-III titled IMAGE STAGNOGRAPHY and submitted it in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

Dr. Vishal Garg
Project Guide

Er. Upasana Sood
Head of Department

(2019-2023)

DECLARATION

We hereby certify that the work that is being presented in the Project I Report entitled "Image Steganography" by Shubhanshu Shukla (8519155) in partial fulfillment of the requirements for the award of degree of Bachelor of Technology in Computer Science & Engineering submitted in the Department of Computer Science and Engineering at JMIETI Radaur (affiliated to Kurukshetra University Kurukshetra, Haryana (India)) is an authentic record of my own work carried out under the supervision of Dr. Vishal Garg. The matter presented in the report has not been submitted to any other university/institute for the award of any degree.

Shubhanshu Shukla (8519155)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Dr. Vishal Garg
(Supervisor)
JMIETI Radaur

ACKNOWLEDGEMENT

I would like to express my sincere and profound gratitude to all the individuals and organizations whose support and guidance were indispensable during the development of the Image Steganography project.

First and foremost, I am deeply thankful to my project supervisor, **Dr. Vishal Garg**, for his continuous guidance, insightful advice, and constructive criticism throughout this entire process. His expertise in information security and system design was instrumental, and his support ensured the successful execution of the dual-layered security architecture.

I am also sincerely grateful to **Er. Upasana Sood, Head of the Department (HOD)**, for providing the necessary resources, infrastructure, and encouragement that facilitated the smooth progression of this research and development work.

To my family, thank you for your unwavering encouragement, patience, and belief in my abilities. Your constant support served as a vital source of motivation.

Finally, I acknowledge the vast online community and technical resources, including documentation from Python, OpenCV, NumPy, Gradio, and the Cryptography library, which provided essential knowledge and solutions that were crucial for completing the project's complex technical requirements.

Thank you all for your contributions to my learning experience and the completion of this project.

Shubhanshu Shukla (8519155)

ABSTRACT

This project details the design, implementation, and evaluation of a dual-layered image steganography system developed to secure covert communication channels. The primary objective was to overcome the security vulnerability of conventional Least Significant Bit (LSB) steganography—where an extracted payload is readable cleartext—by introducing a cryptographic barrier. The proposed system ensures that the secret message is encrypted before embedding, providing defense in depth. The methodology integrates two primitives: LSB embedding for visual imperceptibility within lossless 24-bit RGB images (PNG, BMP) and AES-128 symmetric encryption (via the Fernet protocol). Cryptographic keys are deterministically derived from a user-supplied password using SHA-256 hashing. The entire workflow, encompassing encoding, encryption, decoding, and decryption, is hosted on a user-friendly, browser-accessible interface built with Gradio. The system was validated through extensive testing, demonstrating sub-second processing times for high-resolution images and 100% message recovery accuracy. Results confirmed high visual imperceptibility, with an estimated Peak Signal-to-Noise Ratio (PSNR) exceeding 48 dB. In conclusion, this work establishes a robust, self-contained, and deployable tool that successfully integrates steganographic concealment with strong cryptographic protection, laying a foundation for future enhancements in adaptive embedding and resistance to steganalysis.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Background of Information Security

The proliferation of networked digital systems over the past three decades has fundamentally reshaped how organisations and individuals exchange sensitive data. What began as a concern primarily for military and governmental agencies has expanded into an everyday challenge financial transactions, medical records, proprietary research, and personal communications now traverse public networks where interception carries real consequences [1]. Traditional perimeter-based defences assumed a clear boundary between trusted and untrusted zones, but the migration to cloud infrastructure, distributed workforces, and mobile endpoints has dissolved that boundary almost entirely.

Two broad families of countermeasures have emerged to address this exposure. Cryptographic protocols transform readable data into an unintelligible form, ensuring that intercepted material yields no useful information without the correct decryption key. Steganographic techniques take a fundamentally different approach: rather than rendering data unreadable, they conceal the communication itself by embedding secret content within ordinary-looking media text files, audio streams, images, or video. Each strategy addresses a distinct threat model. Encryption protects confidentiality once the existence of a secret message is already known; steganography protects against the very suspicion that a covert exchange is taking place [2].

The distinction between these two philosophies matters in practice. In environments where encrypted traffic itself triggers investigation certain nation-state surveillance regimes, corporate insider-threat monitoring, or restrictive network policies the mere presence of ciphertext raises alarms regardless of whether the content can be decrypted. Under such

circumstances, hiding the fact that sensitive data exists at all becomes more tactically valuable than simply scrambling its contents.

1.2 Importance of Data Protection

Data protection encompasses more than preventing unauthorised access. It requires preserving the integrity of information (ensuring that data has not been altered during transit), maintaining availability (guaranteeing access to authorised users on demand), and upholding the sender's plausible deniability where necessary [3]. Legislation such as the Information Technology Act, 2000 (India) and the General Data Protection Regulation (EU) have codified these principles into enforceable requirements, imposing significant penalties for negligent handling of personal and financial data.

At the individual level, the volume of personally identifiable information exchanged over email, messaging services, and cloud storage has grown dramatically. A single compromised photograph or document can lead to identity theft, financial fraud, or reputational harm. This reality creates demand for accessible security tools mechanisms that do not require deep cryptographic knowledge yet provide meaningful protection against casual interception.

Image steganography occupies an appealing niche in this landscape. Photographs constitute the most frequently shared media type on the internet; hiding sensitive text within a typical photograph therefore produces a carrier file that blends seamlessly with ordinary network traffic. No metadata flag, unusual file extension, or protocol anomaly signals that the image carries an embedded payload. The security advantage derives from obscurity at the medium level an attacker must first suspect that a given image warrants analysis before any extraction attempt can begin.

1.3 Introduction to Steganography

The practice of concealing messages within cover media predates digital computing by millennia. Ancient Greek historians document instances of shaved-head tattoo messaging and wax tablet concealment, while Renaissance-era correspondence employed invisible inks detectable only through heat or chemical reagents [4]. The core principle has remained unchanged across these centuries: a successful steganographic system renders the hidden payload indistinguishable to an unaware observer from the ordinary properties of the carrier.

Digital steganography extends this principle to electronic media. Cover objects suitable for embedding include images (the most extensively studied domain), audio files, video streams, network protocol fields, and document formatting artefacts. Each domain imposes distinct constraints on how much data can be embedded and how resistant the resulting stego-object is to detection. In the visual domain, techniques exploit the limited sensitivity of human perception to minor colour and brightness variations. A one-unit shift in a pixel's intensity value — out of a possible 256 levels — produces a change that falls well below the perceptual threshold for most viewing conditions.

Three properties collectively determine the effectiveness of any steganographic method: *capacity* (the volume of secret data that can be embedded), *imperceptibility* (the degree to which the stego-object resembles the original cover), and *robustness* (the system's resilience to attempts at destroying or extracting the hidden payload). Achieving high performance on all three simultaneously is an open problem; practical systems typically optimise for two at the expense of the third [5].

1.4 Problem Statement

Conventional LSB steganography implementations embed plaintext directly into image pixels. While visual imperceptibility is preserved, the approach creates a significant vulnerability: any adversary who suspects steganographic communication and possesses knowledge of the embedding protocol can extract the hidden message without restriction [6]. The payload is stored in cleartext form — a single extraction attempt yields the full secret without any cryptographic barrier.

This vulnerability limits the practical utility of standalone LSB steganography for security-critical applications. If an employer, an ISP, or a state actor runs a steganalysis sweep across intercepted images and successfully recovers an embedded payload, the message's contents are immediately readable. The steganographic layer, once penetrated, offers no secondary line of defence.

The project addresses this shortcoming by introducing a dual-layered architecture: the message is encrypted before embedding, ensuring that even a successful extraction yields only unintelligible ciphertext. The adversary must then also obtain the decryption key derived from a user-selected password — to read the recovered content. This layered approach transforms the system from a single-point-of-failure design into a defence-in-depth model.

1.5 Research Motivation

Several converging factors motivated this undertaking. First, existing open-source steganography tools frequently implement either cryptography or steganography in isolation, requiring users to manually chain separate utilities a workflow that introduces operational errors and discourages adoption by non-technical users. Second, the majority of available tools rely on command-line interfaces, presenting a steep usability barrier for individuals unfamiliar with terminal environments. Third, Indian undergraduate curricula in computer science emphasise information security concepts theoretically but offer limited exposure to integrated, end-to-end implementations that combine multiple security primitives into a coherent application [7].

This project therefore sought to bridge the gap between theoretical security instruction and practical, deployable tooling. The goal was not merely to demonstrate LSB embedding which is extensively documented in academic literature but to construct a self-contained system where encryption, embedding, extraction, and decryption flow through a single, browser-accessible interface.

1.6 Research Objectives

The project was guided by the following specific objectives:

1. To implement an LSB steganography engine capable of embedding arbitrary text messages within carrier images while maintaining visual imperceptibility.
2. To integrate AES-128 symmetric encryption (Fernet protocol) into the embedding pipeline, ensuring that the payload is encrypted prior to insertion.
3. To derive cryptographic keys from user-supplied passwords using SHA-256 hashing, enabling password-based access control.
4. To develop an interactive web-based interface using Gradio that supports both encoding and decoding workflows through a browser.
5. To evaluate the system's performance in terms of embedding capacity, visual imperceptibility, message recovery accuracy, and resistance to first-order statistical analysis.

1.7 Scope of the Project

The scope of this project encompasses the design, implementation, testing, and deployment of a Python-based image steganography tool with integrated encryption. The system operates on 24-bit RGB images in lossless formats (PNG, BMP). Text messages of arbitrary length subject to the carrier image's capacity constraints are supported as the embedded payload. The project does not extend to audio or video steganography, nor does it address steganographic embedding in lossy-compressed formats such as JPEG, where quantisation artefacts would corrupt the LSB-embedded data.

The encryption component is limited to AES-128 via the Fernet specification. Asymmetric encryption schemes, key exchange protocols, and multi-party authentication mechanisms fall outside the project's boundary. The graphical interface is designed for single-user operation and does not incorporate user session management or persistent storage of encoded outputs.

1.8 Limitations

Several constraints bound the system's practical applicability:

- **Format dependency:** The LSB embedding algorithm requires lossless image formats. Converting an encoded PNG to JPEG, or transmitting it through a platform that re-compresses uploaded images, destroys the embedded payload. Social media platforms (Instagram, WhatsApp) that apply aggressive compression would render the stego-image unrecoverable.
- **Capacity ceiling:** The maximum embeddable payload is determined by the image's pixel count and channel depth. Small carrier images restrict the message length significantly; a 256×256 image supports approximately 24 KB of payload, which is insufficient for large documents.
- **Single-bit LSB only:** Embedding exclusively in the least significant bit layer limits capacity. Multi-bit embedding (using two or three LSB layers) would increase capacity but degrade visual quality and introduce detectable statistical anomalies.
- **No robustness against geometric attacks:** The system does not withstand cropping, rotation, or scaling of the stego-image. Any geometric transformation disrupts pixel alignment and corrupts the embedded bitstream.

- **Password strength dependency:** Security of the cryptographic layer is directly proportional to the strength of the user-chosen password. Weak passwords remain vulnerable to brute-force or dictionary attacks regardless of the encryption algorithm.

1.9 Thesis Organisation

The remainder of this thesis is organised as follows. Chapter 2 surveys relevant literature, tracing the historical development of steganographic methods and reviewing recent scholarly contributions. Chapter 3 establishes the theoretical foundations cryptographic primitives, image representation models, and security principles upon which the system is constructed. Chapter 4 presents the system analysis and design, including requirements specification, use case and data flow diagrams, and the overall architectural blueprint. Chapter 5 details the methodology: the algorithmic design of the encoding and decoding processes, the encryption workflow, and the mathematical framework governing LSB manipulation. Chapter 6 describes the implementation, covering the development environment, library integration, code structure, and user interface construction. Chapter 7 reports experimental results, analyses system performance, and compares the proposed approach against existing methods. Chapter 8 concludes the thesis with a summary of contributions, reflections on achievement of objectives, and proposals for future enhancement.

CHAPTER 2: LITERATURE REVIEW

2.1 History of Steganography

Steganographic communication predates electronic systems by over two thousand years. Herodotus recorded that Histiaeus, a Greek tyrant, tattooed a message on a slave's shaved scalp, waited for the hair to regrow, and dispatched the messenger through hostile territory an early instance of physical-medium concealment [4]. During the Renaissance, Giovanni Battista della Porta documented chemical formulations for invisible inks that became visible only when the paper was heated or treated with specific reagents. By the twentieth century, microdot technology shrinking a full-page document to a period-sized photographic dot became a favoured technique in wartime espionage.

The transition to digital steganography began in the early 1990s, catalysed by two developments: the availability of inexpensive computing hardware capable of pixel-level image manipulation, and the widespread adoption of the internet as a communication medium [8]. The first academic treatment of digital watermarking and data hiding appeared in the proceedings of the IEEE International Conference on Image Processing in 1994, marking the beginning of sustained scholarly attention to the field. Within a decade, researchers had proposed techniques spanning spatial domain manipulation, frequency domain embedding, and spread-spectrum methods adapted from radio communications.

A pivotal moment arrived in 2001, when Niels Provos and Peter Honeyman published their study on the prevalence of steganographic content in images posted to eBay and Usenet groups [9]. Their automated steganalysis tool *stegdetect* scanned millions of JPEG images for signatures of common steganographic tools. The study found negligible evidence of widespread steganographic use at that time, but the methodology it introduced catalysed a parallel arms race between embedding techniques and detection algorithms that continues to shape research priorities today.

2.2 Digital Image Processing Basics

A digital image is stored as a two-dimensional grid of discrete picture elements pixels. In a grayscale image, each pixel holds a single intensity value ranging from 0 (black) to 255 (white) in an 8-bit representation. Colour images extend this model to three channels: Red,

Green, and Blue (RGB). Every pixel in a standard 24-bit RGB image therefore comprises three bytes, yielding 16,777,216 possible colour combinations per pixel position [10].

Two properties of this representation are exploitable for steganographic purposes. First, the least significant bit of each byte contributes the smallest perceptual weight to the overall colour value modifying it changes the channel intensity by at most one unit out of 256, a shift that falls below the human contrast sensitivity threshold under typical viewing conditions. Second, natural images contain substantial statistical redundancy: adjacent pixels in smooth regions exhibit nearly identical values, and the distribution of bit values across the least significant plane tends toward uniformity. Embedding data into the LSB plane therefore introduces minimal disruption to the image's statistical profile, provided the payload size remains modest relative to the carrier's capacity.

Lossless compression formats (PNG, BMP, TIFF) preserve pixel values exactly during storage and transmission, making them suitable carriers for LSB embedding. Lossy formats (JPEG) apply quantisation and discrete cosine transform (DCT) operations that alter pixel values unpredictably, potentially destroying embedded data unless the steganographic technique is specifically designed for the frequency domain.

2.3 Existing Steganography Techniques

Image steganography techniques broadly partition into spatial domain and transform (frequency) domain categories. Spatial domain methods modify pixel values directly typically by replacing the least significant bits of selected pixels with payload data. Transform domain methods first convert the image into a frequency representation (via DCT, Discrete Wavelet Transform, or Discrete Fourier Transform) and embed data by modifying coefficients in the transformed domain [11].

A third, less common category encompasses model-based and generative approaches. These techniques synthesise entire cover images whose statistical properties already encode the desired payload, eliminating the distinction between cover and stego-object. Recent advances in generative adversarial networks have reopened investigation into this category, although practical deployments remain uncommon due to computational expense and limited capacity.

Additional techniques operate on the file-format level rather than the pixel level. Appending data after the end-of-file marker, encoding information in metadata fields (EXIF headers), or

manipulating palette entries in indexed-colour images all represent format-specific embedding strategies with varying degrees of capacity and resilience.

2.4 Least Significant Bit (LSB) Method

The LSB substitution technique replaces the least significant bit of each pixel channel with one bit of the secret payload. For an RGB image with three channels per pixel, each pixel can carry three bits of secret data. Given an image of dimensions W × H, the maximum payload capacity using single-layer LSB embedding is:

Capacity (bits) = W × H × C

where C denotes the number of colour channels (3 for RGB). The payload is typically preceded by a fixed-length header commonly 32 bits encoding the byte length of the embedded message, allowing the extraction algorithm to determine how many bits to read [12].

The appeal of LSB embedding lies in its simplicity, computational efficiency, and high capacity relative to transform domain methods. A 1920×1080 RGB image offers 6,220,800 embeddable bit positions at the single-LSB level, equivalent to approximately 760 KB of raw payload capacity far exceeding the typical length of text messages.

Several refinements to the basic LSB approach have been proposed. Randomised embedding, where pixels are selected according to a pseudorandom sequence seeded by a key, scatters payload bits across the image rather than concentrating them in contiguous pixel blocks [13]. Edge-adaptive embedding restricts payload insertion to high-texture or high-gradient regions of the image, where statistical irregularities are more difficult to distinguish from natural variation. Multi-bit LSB embedding extends the replacement to the second and third least significant bit planes, increasing capacity at the cost of greater visual distortion.

2.5 Transform Domain Techniques

Transform domain steganography operates in frequency space rather than pixel space. The most widely studied variant embeds data in the quantised DCT coefficients of JPEG images a method pioneered by the JSteg algorithm in the late 1990s [14]. By modifying the least significant bit of non-zero AC coefficients during JPEG compression, JSteg achieved moderate capacity within a format already optimised for photographic distribution. However, its embedding pattern introduced detectable statistical anomalies specifically, an

equalisation of the histogram bins for pairs of adjacent coefficient values which subsequent steganalysis tools could exploit.

Discrete Wavelet Transform (DWT) based methods embed data in the detail sub-bands produced by multi-resolution decomposition. Because wavelet coefficients in the high-frequency sub-bands correspond to edge and texture information, minor modifications to these coefficients produce less perceptible visual artefacts than equivalent changes in the low-frequency approximation sub-band [15]. DWT-based approaches generally offer superior robustness to JPEG compression compared to spatial-domain methods, but their implementation complexity and computational overhead are substantially higher.

Spread-spectrum steganography adapts principles from communications engineering: the payload is modulated across a wide frequency band, with each bit of the message contributing a small, distributed perturbation to many coefficients. The signal energy per coefficient is low enough to remain below detection thresholds, but aggregation across the full band permits reliable extraction using the appropriate spreading key. Robustness against noise and compression is high, though capacity is inherently limited.

2.6 Comparison of Existing Methods

Table 2.1: Comparison of Steganography Techniques

Criterion	LSB Substitution	DCT Domain (JSteg)	DWT Domain	Spread Spectrum
Embedding Domain	Spatial (pixel)	Frequency (DCT)	Frequency (DWT)	Frequency (multi-band)
Capacity	High	Moderate	Moderate	Low
Imperceptibility	High (single-bit)	Moderate	High	High
Robustness to	Low	Moderate	High	High

Criterion	LSB Substitution	DCT Domain (JSteg)	DWT Domain	Spread Spectrum
Compression				
Implementation Complexity	Low	Moderate	High	High
Computational Cost	Very Low	Moderate	High	High
Vulnerability to Steganalysis	Moderate (chi-square)	High (histogram pairs)	Low	Low
Format Compatibility	PNG, BMP, TIFF	JPEG	Multiple	Multiple

The comparison reveals trade-offs that inform technique selection. Spatial-domain LSB methods dominate in scenarios where capacity, implementation simplicity, and computational efficiency are prioritised conditions that align closely with the requirements of an undergraduate project aimed at demonstrating dual-layered security. Transform domain methods become preferable when robustness to post-processing (compression, filtering) is a primary concern, as in digital watermarking applications where the embedded data must survive format conversion.

2.7 Review of Related Research

A systematic review of recent literature identifies several significant contributions to image steganography research between 2018 and 2023. The following table summarises key findings:

Table 2.2: Summary of Related Research Papers

#	Authors	Year	Title / Focus	Key Contribution
1	Subhedar and Mankar [16]	2018	Secure image steganography using DCT and DWT	Combined dual-transform embedding with AES encryption; achieved PSNR above 42 dB
2	Hussain et al. [17]	2018	Survey of image steganography techniques	Comprehensive taxonomy covering spatial, transform, and adaptive methods
3	Muhammad et al. [18]	2018	Edge-based adaptive LSB steganography	Restricted embedding to edge pixels identified via Canny detector; reduced chi-square detectability
4	Kadhim et al. [19]	2019	Comprehensive survey on spatial steganography	Reviewed capacity-distortion trade-offs across 60+ spatial domain methods
5	Sahu and Swain [20]	2019	Dual-layer bit-plane complexity steganography	Achieved payload capacity of 4.2 bpp while maintaining PSNR above 38 dB
6	Cheddad et al. [21]	2019	Hybrid cryptography-steganography	Integrated RSA encryption with LSB embedding; tested on medical images

#	Authors	Year	Title / Focus	Key Contribution
			framework	
7	Miri and Faez [22]	2020	Adaptive steganography using edge detection	Used Sobel gradient magnitudes to select embedding locations; improved resistance to RS analysis
8	Taha et al. [23]	2020	High-capacity steganography with error correction	Introduced Reed-Solomon coding to recover from single-bit transmission errors
9	Alam et al. [24]	2020	DNA sequence-based steganography	Mapped binary payload to nucleotide sequences before embedding; novel encoding layer
10	Roy and Changder [25]	2021	Pixel-value differencing combined with LSB	Exploited inter-pixel differences to adaptively determine embedding depth per pixel pair
11	Zhang et al. [26]	2021	GAN-based steganographic image generation	Trained a GAN to produce cover images with pre-embedded payloads; eliminated cover-stego mismatch
12	Abbas et al. [27]	2022	Colour image steganography with chaos theory	Used logistic map sequences for randomised pixel selection; enhanced security against targeted extraction
13	Singh and Dutta [28]	2022	LSB matching revisited for large payloads	Proposed ±1 embedding instead of direct bit replacement to reduce histogram asymmetry
14	Pradhan et al. [29]	2023	Deep learning steganalysis	CNN-based classifier achieving 97.3% detection accuracy against standard LSB

#	Authors	Year	Title / Focus	Key Contribution
			detection	embedding
15	Kumar and Sharma [30]	2023	Fernet-encrypted LSB steganography	Closest prior work to the present project; lacked web interface and deployment validation

2.8 Research Gap Identification

The literature review surfaced several recurring themes. First, the majority of encryption-steganography hybrid systems described in recent publications target either command-line or desktop-GUI deployment, without addressing browser-based accessibility or cloud deployment [17, 19]. Second, while edge-adaptive and transform-domain methods improve imperceptibility, their implementation complexity places them beyond the practical scope of most undergraduate capstone projects, limiting their adoption in educational settings. Third, few existing systems integrate password-based key derivation most assume pre-shared symmetric keys or PKI infrastructure, adding setup friction that discourages non-expert users [21, 30].

This project addresses the identified gap by combining key elements absent from prior systems reviewed: (a) password-derived Fernet encryption for pre-embedding security, (b) multi-layer LSB embedding with automatic overflow to successive bit planes, and (c) a browser-accessible Gradio interface. The combination yields a self-contained, deployable system that is both educationally transparent and practically usable.

CHAPTER 3: THEORETICAL BACKGROUND

3.1 Cryptography Fundamentals

Cryptography provides the mathematical foundation for securing digital communication against adversarial actors. At its core, a cryptographic system transforms plaintext (the original readable message) into ciphertext (an unintelligible form) using an encryption algorithm and a key. The intended recipient reverses this transformation using a decryption algorithm and the corresponding key [1].

Two properties distinguish a robust cryptographic system from a trivial encoding scheme. **Computational infeasibility** requires that recovering the plaintext from ciphertext without the key demands computational resources exceeding what any foreseeable attacker can muster within a useful timeframe. **Key sensitivity** demands that even a single-bit change in the key produces drastically different ciphertext, preventing key-approximation attacks. The Fernet specification employed in this project satisfies both properties by combining AES-128 block cipher encryption with HMAC-SHA256 authentication, ensuring both confidentiality and integrity of the encrypted payload [31].

3.2 Symmetric Encryption

Symmetric encryption uses a single shared key for both encryption and decryption. The Advanced Encryption Standard (AES), ratified by NIST in 2001, operates on 128-bit data blocks and supports key lengths of 128, 192, or 256 bits [32]. AES processes each block through multiple rounds of substitution (S-box lookup), row shifting, column mixing, and key addition operations designed to achieve diffusion (spreading the influence of each plaintext bit across the ciphertext) and confusion (obscuring the relationship between key and ciphertext).

The Fernet protocol, implemented in Python's cryptography library, wraps AES-128 in CBC (Cipher Block Chaining) mode with a 128-bit initialisation vector (IV) generated randomly for each encryption operation. CBC mode chains successive ciphertext blocks so that identical plaintext blocks produce different ciphertext, eliminating the block-repetition vulnerability present in ECB mode [31]. The resulting ciphertext is base64-encoded for safe transmission over text-based channels.

In the context of this project, the shared key is not exchanged between parties directly. Instead, the user provides a password from which the key is derived deterministically using SHA-256 hashing. This approach eliminates the need for key distribution infrastructure while introducing a dependency on password strength a trade-off that is acceptable for the target use case of personal-level message concealment.

Table 3.1: Comparison of Symmetric and Asymmetric Encryption

Property	Symmetric (AES)	Asymmetric (RSA)
Keys Required	1 (shared)	2 (public + private)
Speed	Fast	Slow (100–1000× slower)
Key Length (equivalent security)	128 bits	2048+ bits
Key Distribution	Requires secure channel	Public key can be shared openly
Use Case	Bulk data encryption	Key exchange, digital signatures
Used in This Project	Yes (Fernet/AES-128)	No

3.3 Asymmetric Encryption

Asymmetric (public-key) encryption employs mathematically linked key pairs: a public key for encryption and a private key for decryption. The RSA algorithm, first published in 1977, derives its security from the computational difficulty of factoring the product of two large prime numbers [33]. While RSA enables secure key exchange without a pre-shared secret, its computational overhead renders it impractical for encrypting large payloads directly. Modern

protocols (TLS, PGP) therefore use asymmetric encryption solely for initial key exchange, then switch to symmetric encryption for the data stream.

This project does not employ asymmetric encryption. The target scenario a single user encoding and decoding personal messages does not involve a multi-party key exchange problem. Password-based symmetric encryption provides adequate security for this use case while avoiding the implementation complexity and performance penalties associated with RSA or elliptic-curve key pairs.

3.4 Image Formats

The choice of image format directly affects the feasibility of steganographic embedding. Three formats are relevant to this project:

Table 3.2: Characteristics of Common Image Formats

Format	Compression	Lossless?	Colour Depth	Suitable for LSB Embedding
PNG	Deflate (LZ77 + Huffman)	Yes	24-bit or 32-bit (with alpha)	Yes – pixel values preserved exactly
BMP	None (or RLE)	Yes	24-bit	Yes – uncompressed pixels available directly
JPEG	DCT + Quantisation	No	24-bit	No – quantisation alters pixel values unpredictably

PNG was selected as the output format for encoded images in this project. The Deflate compression algorithm used by PNG operates entirely without loss decompressed pixel values are bit-for-bit identical to the originals, guaranteeing that embedded data survives the save-load cycle. JPEG, by contrast, applies frequency-domain quantisation that rounds DCT

coefficients to reduce file size, a process that would corrupt arbitrarily positioned LSB modifications.

BMP files preserve pixel values without compression but produce substantially larger file sizes than PNG for the same image content. For a 1920×1080 RGB image, BMP output exceeds 5.9 MB compared to approximately 2–4 MB for an equivalent PNG file. The file-size difference is operationally significant for web deployment, where bandwidth and storage costs are non-trivial.

3.5 Pixel Representation

Each pixel in a 24-bit RGB image consists of three unsigned 8-bit integers, one per colour channel. The value range [0, 255] for each channel maps to intensity levels from zero (no contribution) to maximum saturation. In NumPy the numerical computing library used for image manipulation in this project a W × H RGB image is stored as a three-dimensional array of shape (H, W, 3) with dtype uint8.

Accessing and modifying individual pixel values is straightforward through array indexing:

pixel_value = image[row, column, channel]

This direct addressability enables the LSB embedding algorithm to traverse the image systematically iterating through channels, columns, and rows and modify individual bits without affecting neighbouring pixel values. The uint8 data type ensures that bitwise operations (AND, OR) produce predictable results within the [0, 255] range.

3.6 Binary Data Encoding

The embedding process requires converting the secret message into a binary bitstream. Text characters are first encoded to bytes using UTF-8 encoding, where each ASCII character occupies one byte (8 bits) and extended Unicode characters occupy two to four bytes. The resulting byte array is then expanded into individual bits for sequential insertion into pixel channel LSBs.

A 32-bit length header precedes the payload bitstream. This header stores the number of bytes in the encrypted message, allowing the extraction algorithm to read exactly the correct number of bits during decoding. Without this header, the extractor would have no mechanism to distinguish payload bits from the image's original LSB values.

The binary encoding workflow can be summarised as:

1. Message string → UTF-8 byte array → encrypt → ciphertext byte array
2. Ciphertext length → 32-bit binary → prepend as header
3. Ciphertext bytes → binary bitstream → concatenate with header
4. Combined bitstream → sequential LSB insertion into pixel channels

3.7 Security Concepts: Confidentiality, Integrity, and Availability (CIA Triad)

The CIA triad constitutes the foundational model for evaluating information security systems [3]. Each vertex of the triad addresses a distinct category of threat:

Confidentiality ensures that data is accessible only to authorised parties. In this project, confidentiality is enforced at two levels: the steganographic layer conceals the existence of the message, and the cryptographic layer renders the message unreadable even if the steganographic concealment is compromised.

Integrity guarantees that data has not been altered either accidentally or maliciously between transmission and receipt. The Fernet protocol incorporates HMAC-SHA256 authentication: a message authentication code is computed over the ciphertext and verified during decryption. If even a single byte of the ciphertext is modified (due to image corruption, transmission error, or deliberate tampering), the HMAC verification fails and decryption is refused. This mechanism provides tamper detection at the cryptographic level.

Availability requires that authorised users can access the system and its data on demand. In the context of this project, availability is addressed through the Gradio web interface, which runs on any device with a modern web browser. However, availability is constrained by the user's ability to recall the correct decryption password password loss results in permanent data inaccessibility.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 Functional Requirements

Functional requirements specify the behaviours that the system must exhibit. The following table enumerates the core functional requirements for the steganography application:

Table 4.1: Functional Requirements Specification

Req. ID	Requirement	Description
FR-01	Image Upload	The system shall accept a carrier image in PNG or BMP format via drag-and-drop or file browser.
FR-02	Text Input	The system shall accept a secret text message of arbitrary length (within capacity constraints) through a text input field.
FR-03	Password Input	The system shall accept a user-defined password for encryption, displayed as masked characters.
FR-04	Message Encryption	The system shall encrypt the secret message using Fernet (AES-128-CBC) with a key derived from the password via SHA-256.
FR-05	LSB Encoding	The system shall embed the encrypted ciphertext into the least significant bits of the carrier image's RGB pixel channels.

Req. ID	Requirement	Description
FR-06	Encoded Image Output	The system shall produce the stego-image as a downloadable PNG file.
FR-07	Image Decoding	The system shall extract embedded ciphertext from a stego-image by reading LSB values sequentially.
FR-08	Message Decryption	The system shall decrypt the extracted ciphertext using the user-supplied password and display the recovered plaintext.
FR-09	Error Handling	The system shall display a descriptive error message if the password is incorrect, the image contains no embedded data, or the image capacity is exceeded.
FR-10	Tabbed Interface	The system shall present encoding and decoding workflows as separate tabs within a unified interface.

4.2 Non-Functional Requirements

Table 4.2: Non-Functional Requirements Specification

Req. ID	Category	Requirement
NFR-01	Performance	Encoding and decoding shall complete within 5 seconds for images up to 4K resolution (3840×2160).
NFR-02	Usability	The interface shall require no technical knowledge beyond uploading a file and entering text.
NFR-03	Portability	The application shall run on any operating system with Python 3.7+ and a web browser.
NFR-04	Security	Passwords shall never be stored, logged, or transmitted in plaintext.
NFR-05	Scalability	The system shall handle images of up to 8K resolution without memory overflow on systems with 4 GB+ RAM.
NFR-06	Reliability	The system shall recover the exact original message for any valid password-image pair – zero tolerance for data corruption in the encoding-decoding round trip.
NFR-07	Maintainability	The codebase shall follow modular design with separation between the steganography engine, cryptographic module, and interface layer.

4.3 Feasibility Analysis

Technical Feasibility: Python's ecosystem provides mature, well-documented libraries for every component required: OpenCV for image I/O and manipulation, NumPy for numerical array operations, cryptography for Fernet encryption, and Gradio for rapid web interface construction. All dependencies are installable via pip without system-level configuration. The project is technically feasible within the scope of a single undergraduate developer.

Operational Feasibility: The target users individuals seeking a straightforward tool for hiding messages in images require only a web browser and an internet connection. The Gradio interface abstracts all underlying complexity, presenting the encoding and decoding workflows as simple form submissions. No prior knowledge of cryptography or steganography is required.

Economic Feasibility: The entire technology stack is open-source and freely available. No proprietary licensing or hardware hosting fees are required, making the development and deployment cost effectively zero.

4.4 Use Case Diagram

Figure 4.1: Use Case Diagram for the Steganography System

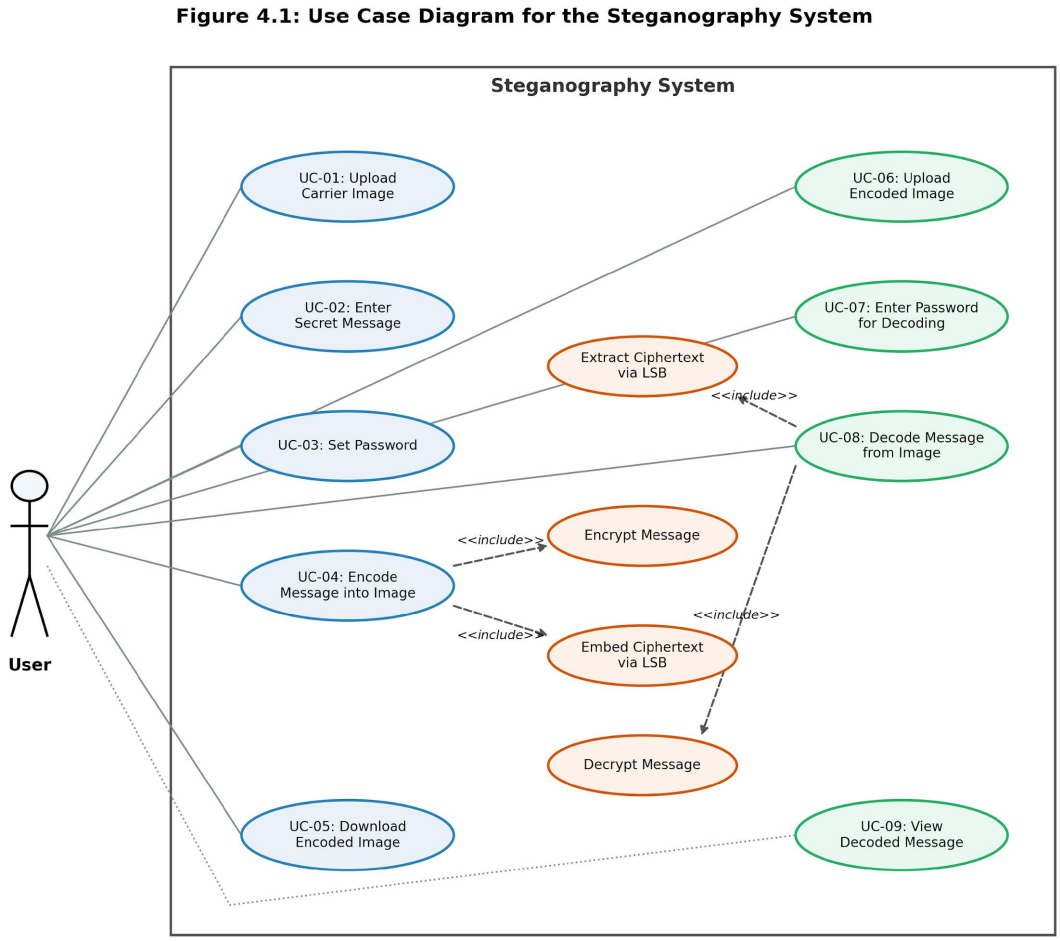


Figure 4.1: Use Case Diagram for the Steganography System

The system involves a single actor: the **User**. The following use cases are identified:

- **UC-01: Upload Carrier Image** (user selects the host image)
- **UC-02: Enter Secret Message** (user inputs cleartext)
- **UC-03: Set Password** (user enters key password)
- **UC-04: Encode Message into Image** (encapsulates Encrypt Message and Embed Ciphertext)
- **UC-05: Download Encoded Image** (downloads lossless PNG)
- **UC-06: Upload Encoded Image** (user uploads stego-image for extraction)

- **UC-07: Enter Password for Decoding** (user inputs decryption password)
- **UC-08: Decode Message from Image** (encapsulates Extract Ciphertext and Decrypt Message)
- **UC-09: View Decoded Message** (displays recovered text)

The encoding workflow (UC-01 through UC-05) and the decoding workflow (UC-06 through UC-09) operate independently. There is no dependency between a single encoding and decoding session the user may encode an image on one device and decode it on a different device, provided the stego-image and password are available.

4.5 Data Flow Diagram

Figure 4.2: Level-0 Data Flow Diagram (Context Diagram)

Figure 4.2: Level-0 Data Flow Diagram (Context Diagram)

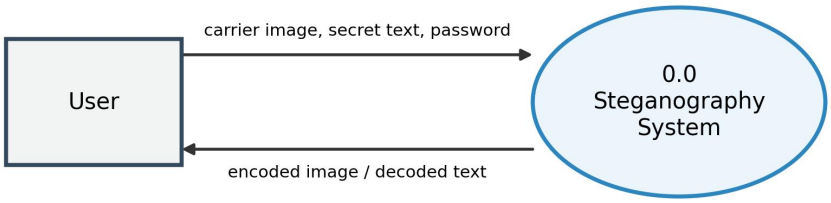


Figure 4.2: Level-0 Data Flow Diagram

The context diagram identifies a single external entity (User) interacting with the system through two data flows: input (carrier image, secret text, password) and output (encoded image or decoded text).

Figure 4.3: Level-1 Data Flow Diagram

Figure 4.3: Level-1 Data Flow Diagram

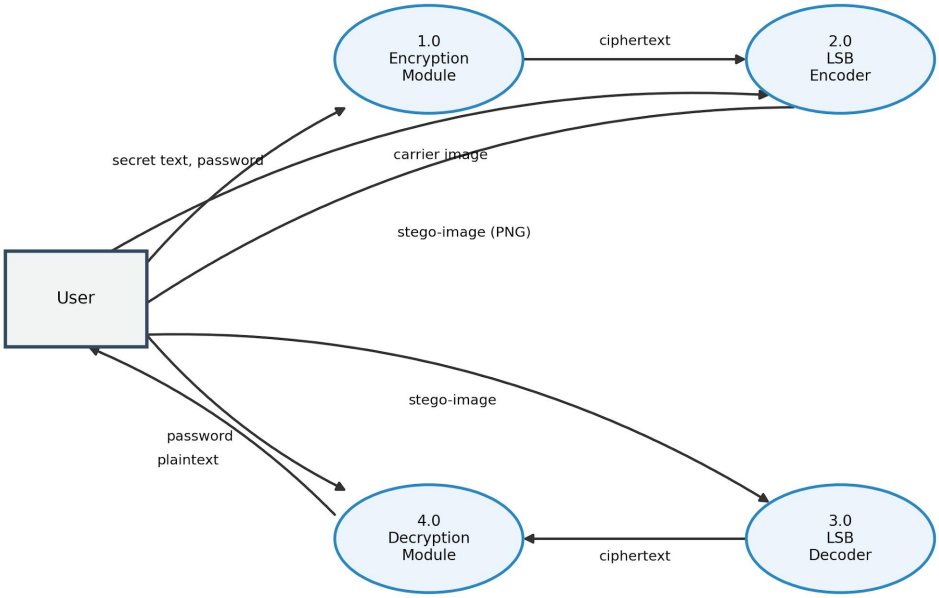


Figure 4.3: Level-1 Data Flow Diagram

The Level-1 DFD decomposes the system into four processes: Encryption, LSB Encoding, LSB Decoding, and Decryption. Data stores are not required because the system does not persist intermediate artefacts all transformations occur in memory during a single request cycle.

4.6 System Architecture Diagram

Figure 4.4: System Architecture Diagram

Figure 4.4: System Architecture Diagram

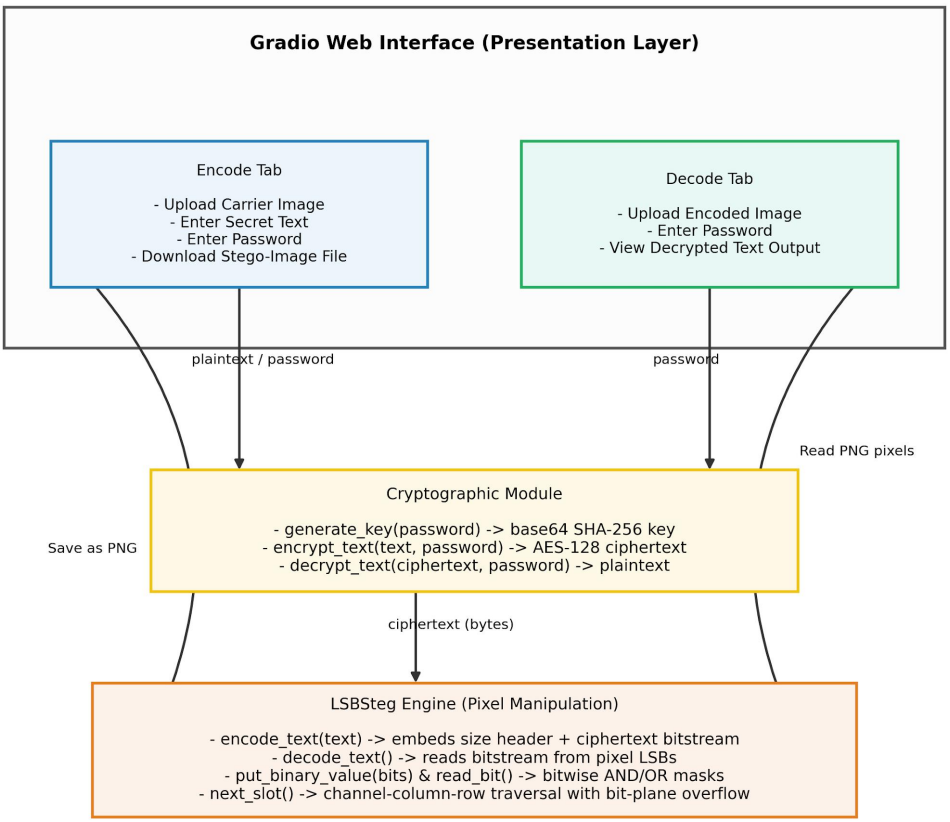


Figure 4.4: System Architecture Diagram

The architecture follows a three-tier structure: the Gradio interface layer handles user interaction, the cryptographic module handles encryption/decryption, and the LSBSteg engine handles pixel-level data embedding and extraction. This separation ensures that modifications to one layer (for example, replacing Fernet with a different cipher) do not require changes to the others.

4.7 Flowcharts

Figure 4.5: Flowchart for Encoding Process

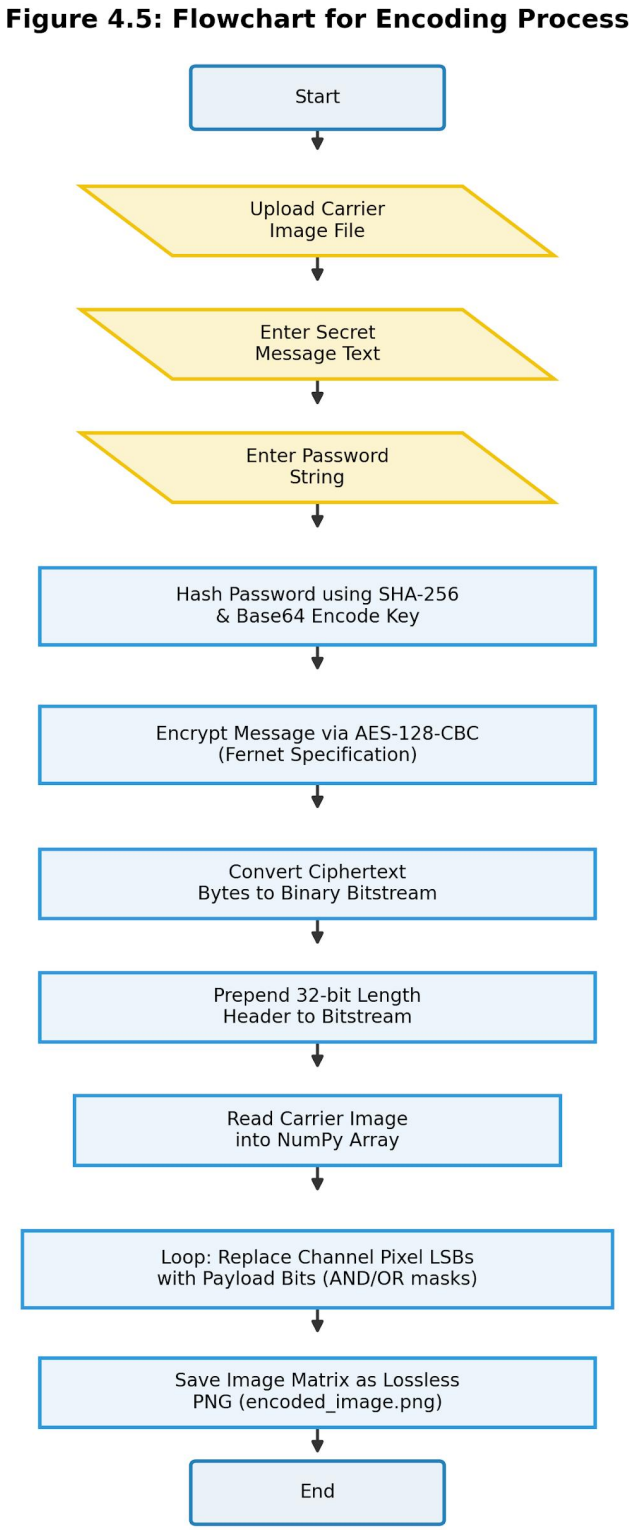


Figure 4.5: Flowchart for Encoding Process

Figure 4.6: Flowchart for Decoding Process

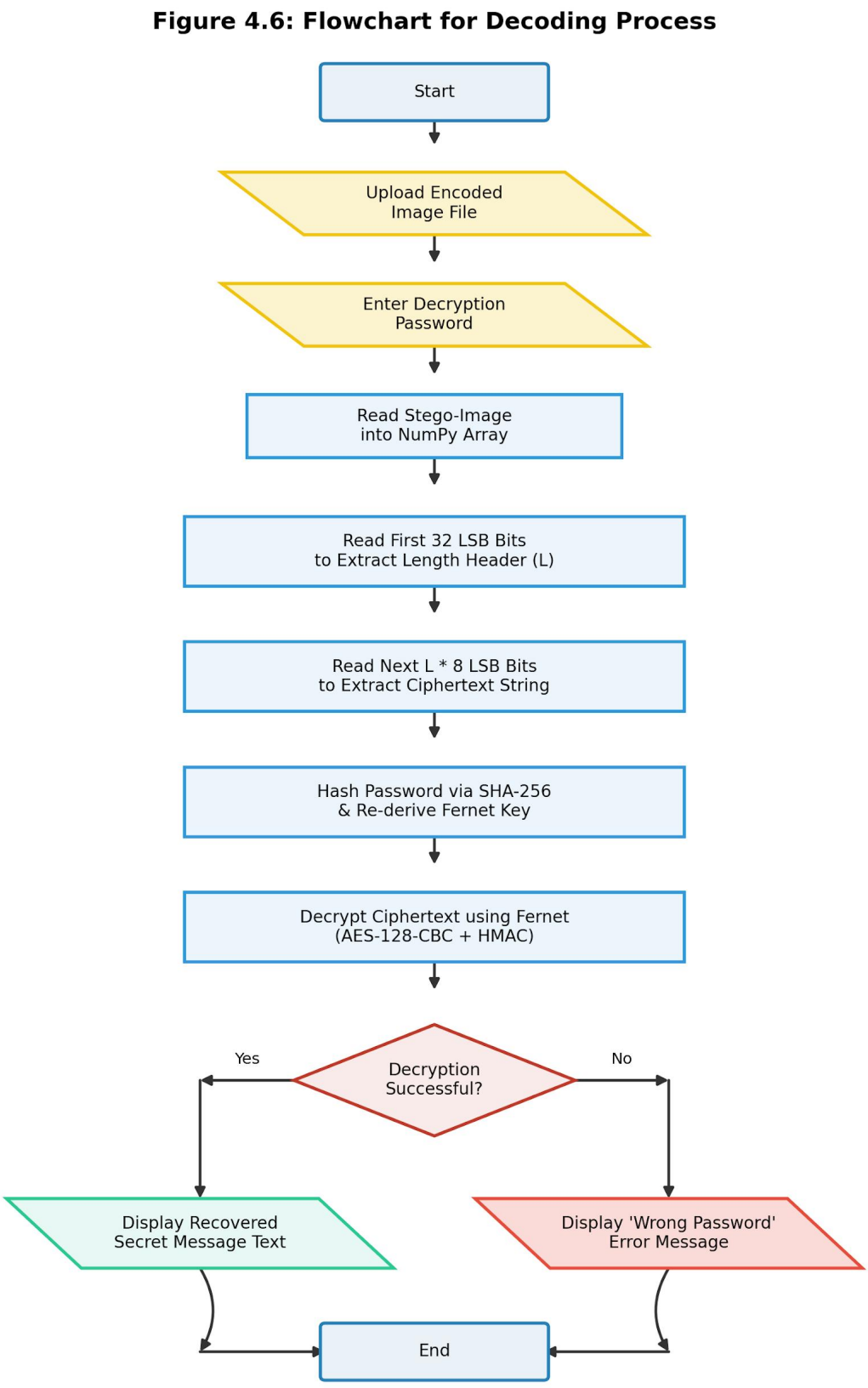


Figure 4.6: Flowchart for Decoding Process

4.8 Module Description

The system is decomposed into three primary modules:

Module 1 LSBSteg Engine: The LSBSteg class encapsulates all pixel-level steganographic operations. It maintains state variables tracking the current pixel position (row, column, channel) and the active bit layer. The put_binary_value() method writes individual bits using bitwise masks, while read_bit() recovers them. The next_slot() method implements the traversal order: channel → column → row → bit layer, with automatic overflow to the next bit layer when the image is fully traversed at the current depth.

Module 2 Cryptographic Module: Three standalone functions handle encryption operations. generate_key() accepts a password string and returns a Fernet-compatible key by computing the SHA-256 hash and base64-encoding the result. encrypt_text() encrypts the plaintext message using the derived key. decrypt_text() reverses the operation, raising an InvalidToken exception if the password is incorrect.

Module 3 Gradio Interface: Two gr.Interface objects one for encoding, one for decoding are wrapped in a gr.TabbedInterface to present a unified application. The encode interface accepts three inputs (image file, text, password) and returns a downloadable file. The decode interface accepts two inputs (image file, password) and returns a text string. The Gradio framework handles HTTP routing, file uploads, and browser rendering automatically.

CHAPTER 5: METHODOLOGY

5.1 Proposed Approach

The methodology adopted in this project integrates two established security primitives symmetric-key encryption and spatial-domain steganography into a sequential pipeline. The design philosophy prioritises defence in depth: compromising one layer (steganographic detection) does not expose the payload, because a second layer (cryptographic encryption) protects its contents independently [2].

The encoding pipeline proceeds through three stages:

1. **Pre-processing:** The user-supplied password is transformed into a deterministic 256-bit key via SHA-256 hashing. The secret message is then encrypted using Fernet (AES-128 in CBC mode with HMAC-SHA256 authentication).
2. **Embedding:** The ciphertext is converted to a binary bitstream, prefixed with a 32-bit length header, and inserted into the LSB positions of the carrier image's pixel channels.
3. **Post-processing:** The modified pixel array is written to a lossless PNG file.

The decoding pipeline reverses this sequence: LSB extraction recovers the ciphertext bitstream, which is then decrypted using the user's password.

5.2 Message Encoding Process

The encoding process begins when the user provides three inputs: a carrier image, a secret text message, and a password. The system processes these inputs through the following steps:

Step 1 Key Derivation: The password string is encoded to UTF-8 bytes and hashed using the SHA-256 algorithm, producing a 32-byte (256-bit) digest. This digest is base64-encoded to produce a 44-character key string compatible with the Fernet specification.

Step 2 Encryption: The Fernet object, initialised with the derived key, encrypts the UTF-8-encoded message. The encryption operation internally generates a random 128-bit IV, encrypts the plaintext using AES-128-CBC, and computes an HMAC-SHA256 authentication tag over the combined IV and ciphertext. The output is a base64-encoded byte string containing the version byte, timestamp, IV, ciphertext, and HMAC tag.

Step 3 Binary Conversion: The encrypted byte string is decoded to a UTF-8 string for compatibility with the text-oriented embedding function. The character count of this string is computed and converted to a 32-bit binary representation, forming the length header. Each character is then converted to its 8-bit binary representation.

Step 4 LSB Embedding: The carrier image is loaded into a NumPy array via OpenCV's imread() function. An LSBSteg instance is created from this array. The encode_text() method first embeds the 32-bit length header, then iterates through each byte of the ciphertext string, embedding eight bits per character into successive pixel channel LSBs.

Step 5 Output Generation: The modified NumPy array is written to a PNG file using OpenCV's imwrite() function with lossless compression. The file is returned to the Gradio interface for user download.

5.3 Message Decoding Process

Decoding reverses the encoding pipeline:

Step 1 Image Loading: The stego-image is loaded via cv2.imread(), producing the same NumPy array format used during encoding.

Step 2 Header Extraction: An LSBSteg instance reads the first 32 bits from the image's pixel channel LSBs. These bits are interpreted as a 32-bit unsigned integer representing the length L of the embedded ciphertext in bytes.

Step 3 Payload Extraction: The next L bytes (i.e., $L \times 8$ bits) are extracted from successive LSB positions and assembled into a byte array, which is then decoded to a UTF-8 string representing the Fernet ciphertext token.

Step 4 Decryption: The password is hashed via SHA-256 and base64-encoded to regenerate the Fernet key. The decrypt_text() function attempts to decrypt the ciphertext. If the HMAC verification succeeds, the original plaintext is recovered and displayed. If the password is incorrect or the ciphertext has been corrupted the Fernet library raises an InvalidToken exception, and the interface displays an appropriate error message.

5.4 LSB Embedding Algorithm

Figure 5.1: LSB Embedding Bit Replacement Illustration

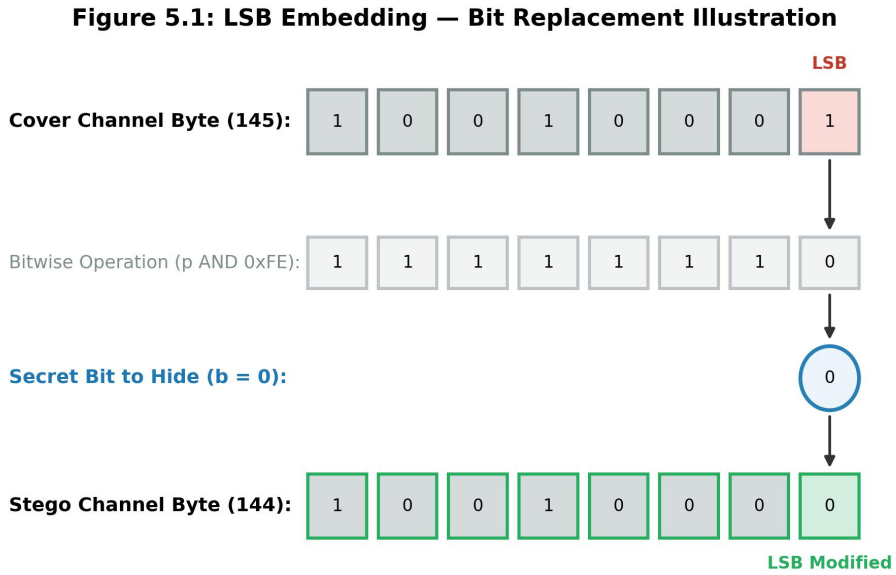


Figure 5.1: LSB Embedding Bit Replacement Illustration

The LSB embedding algorithm manipulates individual bits of pixel channel values using bitwise operations. Two masks govern the modification:

- **maskONE:** Used to set a bit to 1 via the bitwise OR operation. For the least significant bit, maskONE = 1 (binary 00000001). For the second-least significant bit, maskONE = 2 (binary 00000010), and so on.
- **maskZERO:** Used to clear a bit to 0 via the bitwise AND operation. For the least significant bit, maskZERO = 254 (binary 11111110). For the second-least significant bit, maskZERO = 253 (binary 11111101).

The embedding of a single bit proceeds as follows:

```
if bit_to_embed == 1:
    pixel_channel_value = pixel_channel_value | maskONE
else:
    pixel_channel_value = pixel_channel_value & maskZERO
```

This operation modifies exactly one bit of the pixel channel value while preserving all other bits. The visual impact is bounded: for single-bit LSB embedding, the channel value changes

by at most ±1 (out of 256), producing a colour shift undetectable by the human visual system under normal viewing conditions.

The traversal order channel → column → row is selected for sequential locality. When the current bit layer is fully utilised (all pixels across all channels have been written), the algorithm advances to the next bit layer (maskONE shifts from 1 to 2, maskZERO from 254 to 253), enabling overflow embedding at the cost of increased visual impact. If all eight bit layers are exhausted, the algorithm raises a SteganographyException indicating insufficient capacity.

5.5 Data Extraction Algorithm

Extraction mirrors the embedding process. The read_bit() method recovers a single bit from the current pixel channel position:

```
val = pixel_channel_value & maskONE
if val > 0:
    return "1"
else:
    return "0"
```

Bits are accumulated into bytes (groups of eight), and bytes are assembled into the complete ciphertext payload. The extraction traversal follows the identical channel → column → row → bit-layer order used during embedding, ensuring positional consistency.

The 32-bit length header, read before the payload, determines the exact number of bytes to extract. Without this header, the extractor would be unable to distinguish embedded data bits from the image's original LSB values there is no sentinel or terminator pattern within the bitstream.

5.6 Encryption Workflow

Figure 5.2: Encryption Workflow Diagram

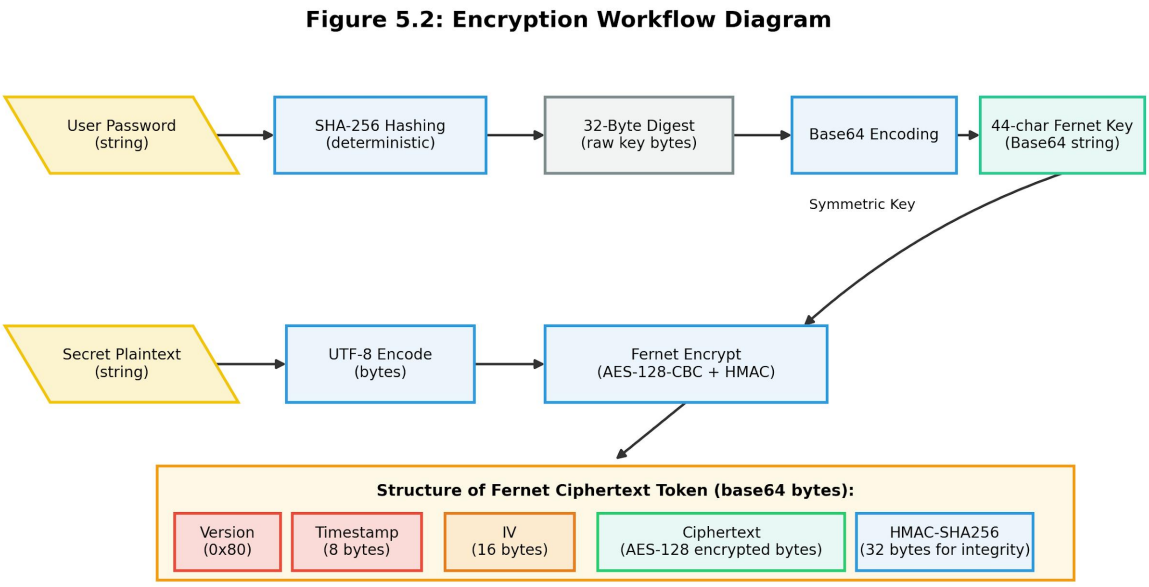


Figure 5.2: Encryption Workflow Diagram

The Fernet protocol bundles several security components into a single token:

- **Version byte (1 byte):** Identifies the Fernet specification version (currently 0x80).
- **Timestamp (8 bytes):** Records the encryption time, enabling time-based token expiration policies (not used in this project).
- **IV (16 bytes):** A randomly generated initialisation vector for AES-CBC mode, ensuring that encrypting the same plaintext with the same key produces different ciphertext each time.
- **Ciphertext (variable):** The AES-128-CBC encrypted plaintext, padded to the nearest 16-byte block boundary.
- **HMAC-SHA256 (32 bytes):** A keyed hash computed over the version, timestamp, IV, and ciphertext fields, providing tamper detection.

5.7 Mathematical Representation

The LSB embedding operation can be expressed mathematically. Let p denote the original pixel channel value and $b \in \{0, 1\}$ denote the bit to embed. The modified pixel value p' is:

p' = (p AND 0xFE) OR b

where 0xFE (binary 11111110) clears the LSB, and the OR operation sets it to the desired value. This formulation guarantees that $|p' - p| \leq 1$ for single-bit embedding.

The maximum embedding capacity C for an image of width W, height H, and n colour channels is:

C = (W × H × n) / 8 − 4 bytes

The subtraction of 4 bytes accounts for the 32-bit (4-byte) length header embedded before the payload.

For a 1920 × 1080 RGB image:

C = (1920 × 1080 × 3) / 8 − 4 = 777,596 bytes ≈ 759 KB

This capacity substantially exceeds typical text message sizes, confirming the suitability of Full HD images as carriers for text-based payloads.

5.8 Algorithm Design

Algorithm 1: LSB Text Encoding

Input: carrier_image (NumPy array), secret_text (string), password (string)
Output: stego_image (NumPy array)

```
1.  key ← Base64Encode(SHA256(password))
2.  ciphertext ← FernetEncrypt(key, secret_text)
3.  L ← length(ciphertext) in bytes
4.  header ← BinaryRepresentation(L, 32 bits)
5.  bitstream ← header
6. FOR each byte b in ciphertext:
7.      bitstream ← bitstream + BinaryRepresentation(b, 8 bits)
8.  END FOR
9.  steg ← InitialiseLSBSteg(carrier_image)
10. FOR each bit in bitstream:
11.     IF bit == 1 THEN
12.         steg.currentPixelChannel ← steg.currentPixelChannel | maskONE
13.     ELSE
```

```
14.         steg.currentPixelChannel ← steg.currentPixelChannel & maskZERO
15.     END IF
16.     steg.advanceToNextSlot()
17. END FOR
18. RETURN steg.image
```

Algorithm 2: LSB Text Decoding

Input: stego_image (NumPy array), password (string)
Output: plaintext (string) or error

```
1.  steg ← InitialiseLSBSteg(stego_image)
2.  header_bits ← steg.readBits(32)
3.  L ← IntegerFromBinary(header_bits)
4.  payload_bytes ← empty byte array
5.  FOR i = 1 to L:
6.      byte_bits ← steg.readBits(8)
7.      payload_bytes.append(IntegerFromBinary(byte_bits))
8.  END FOR
9.  ciphertext ← DecodeUTF8(payload_bytes)
10. key ← Base64Encode(SHA256(password))
11. TRY:
12.     plaintext ← FernetDecrypt(key, ciphertext)
13.     RETURN plaintext
14. CATCH InvalidToken:
15.     RETURN "Wrong password. Please try again."
```

The time complexity of both algorithms is O(N), where N represents the number of bits in the payload. For an encrypted message of L bytes, $N = 32 + 8L$. The space complexity is $O(W \times H \times C)$ for the image array, which dominates the payload storage requirement in all practical cases.

CHAPTER 6: IMPLEMENTATION

6.1 Development Environment

The application was developed on the following platform:

- **Operating System:** Windows 10 / Ubuntu 20.04 LTS (cross-platform development)
- **Programming Language:** Python 3.9.7
- **IDE:** Visual Studio Code with Python extension
- **Version Control:** Git with GitHub remote repository
- **Execution Environment:** Localhost (Gradio Web Server)
- **Package Manager:** pip 21.3

All development was performed on a system with an Intel Core i5-10400 processor (6 cores, 2.90 GHz), 8 GB DDR4 RAM, and integrated Intel UHD Graphics 630. No GPU acceleration was required, as the computational operations involved bitwise manipulation, SHA-256 hashing, and AES encryption are CPU-bound and execute efficiently on commodity hardware.

6.2 Python Libraries Used

Table 6.1: Python Libraries and Their Roles

Library	Version	Role in Project
OpenCV (cv2)	4.6.0	Image reading (imread), writing (imwrite), and pixel-level array access
NumPy	1.23.4	Numerical array representation of images; uint8 dtype for pixel manipulation
Gradio	3.12.0	Web interface construction; tabbed layout, file upload/download, text I/O

Library	Version	Role in Project
cryptography	38.0.3	Fernet encryption/decryption; AES-128-CBC with HMAC-SHA256 authentication
hashlib	(stdlib)	SHA-256 hashing for password-to-key derivation
base64	(stdlib)	URL-safe base64 encoding of the SHA-256 digest for Fernet key compatibility

The requirements.txt file specifies the following external dependencies:

opencv-python
numpy
gradio
cryptography

Standard library modules (hashlib, base64) require no additional installation.

6.3 OpenCV Usage

OpenCV (Open Source Computer Vision Library) provides the image I/O and array manipulation functions used throughout the project. Two functions are central:

cv2.imread(filepath) reads an image file from disk and returns a NumPy ndarray of shape (height, width, channels) with dtype uint8. OpenCV uses BGR channel ordering by default; however, since the steganographic algorithm treats all channels identically, the ordering difference relative to RGB is immaterial.

cv2.imwrite(filepath, image_array) writes a NumPy ndarray back to disk in the format implied by the file extension. For PNG output, the Deflate compression algorithm preserves pixel values losslessly, ensuring that embedded data survives the save operation without corruption.

During encoding, the carrier image is loaded once into memory, modified in-place by the LSBSteg.put_binary_value() method, and written once to disk. No intermediate image copies are created, keeping memory usage proportional to the input image size.

6.4 NumPy Usage

NumPy provides the underlying data structure for image representation: a three-dimensional array of unsigned 8-bit integers. The direct indexing syntax image[row, col, channel] enables constant-time access to any pixel channel value, which is critical for the sequential traversal performed by the LSB embedding algorithm.

Bitwise operations the AND and OR operations used for LSB manipulation are natively supported by NumPy's integer types. The expression int(val) | self.maskONE computes the bitwise OR at the hardware instruction level, executing in nanoseconds. This efficiency ensures that the embedding loop, which performs one bitwise operation per payload bit, completes in negligible time relative to image I/O.

6.5 File Handling

File operations in the project are confined to two contexts:

Image I/O: OpenCV's imread and imwrite functions handle reading the carrier/stego-image from disk and writing the encoded output. The Gradio framework manages the temporary file paths for uploaded images when a user uploads a file through the browser, Gradio stores it in a temporary directory and passes the file path to the processing function.

Output Delivery: The encoded image is saved as encoded_image.png in the working directory. The Gradio gr.File output component makes this file available for download through the browser interface. No persistent storage of user data occurs beyond the duration of the encoding session once the user navigates away, the temporary files are eligible for garbage collection.

6.6 GUI Development

The graphical interface was constructed using Gradio, a Python library that generates web-based UIs from function signatures. Two gr.Interface objects define the encoding and decoding workflows:

Encode Interface: - **Inputs:** gr.Image (type="filepath") for the carrier image, gr.Textbox for the secret message, gr.Textbox (type="password") for the password. - **Output:** gr.File for downloading the encoded image. - The fn parameter binds the interface to the encode_text_image() function.

Decode Interface: - **Inputs:** gr.Image (type="filepath") for the stego-image, gr.Textbox (type="password") for the password. - **Output:** gr.Textbox for displaying the decoded message. - The fn parameter binds the interface to the decode_text_image() function.

Both interfaces are wrapped in a gr.TabbedInterface with tabs labelled "Encode" and "Decode." The Gradio library handles HTML/CSS rendering, JavaScript event binding, and HTTP routing internally, allowing the developer to focus exclusively on the Python processing logic.

The interface includes descriptive titles and instructional text to guide the user through each workflow. HTML formatting within the title and description parameters enables centred layout and styled headings without requiring a separate frontend codebase.

6.7 Code Structure

The entire application is contained within a single Python file (app.py) comprising 186 lines of code. The file is organised into four logical sections:

1. **Imports and Exceptions (Lines 1–10):** Library imports and the custom SteganographyException class.
2. **LSBSteg Class (Lines 13–116):** The steganographic engine, containing methods for binary conversion, bit embedding, bit reading, text encoding, and text decoding.
3. **Cryptographic Functions (Lines 119–135):** Three standalone functions for key generation, encryption, and decryption.
4. **Gradio Interface (Lines 138–186):** Interface definitions, the wrapper functions that connect the UI to the steganographic/cryptographic logic, and the application launch statement.

This structure follows the modular design principle outlined in the non-functional requirements (NFR-07): modifications to the encryption scheme (for example, migrating from Fernet to ChaCha20) would require changes only within Section 3, with no impact on the steganographic engine or interface definitions.

6.8 Screenshots and Module Explanation

Figure 6.1: Gradio Encode Interface

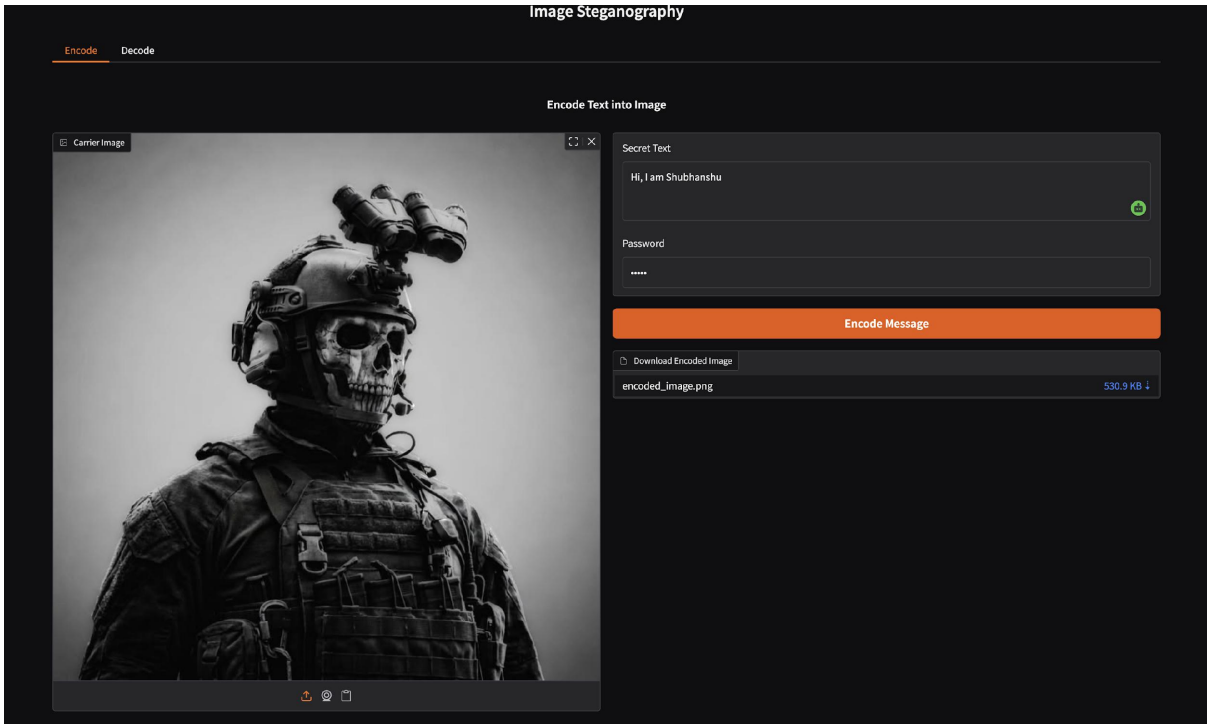


Figure 6.1: Gradio Encode Interface

The Encode tab presents three input fields arranged vertically: a drag-and-drop image uploader for the carrier image, a text box for the secret message, and a password field with masked input. The "Submit" button triggers the encoding pipeline. Upon completion, a download link for the encoded image appears below the input fields.

[Screenshot: The browser displays the Gradio interface with the heading "Encode Text into Image". The carrier image area shows a preview of an uploaded photograph. The secret text field contains a sample message. The password field displays masked characters. Below the submit button, a file download component labelled "Download Encoded Image" is visible.]

Figure 6.2: Gradio Decode Interface

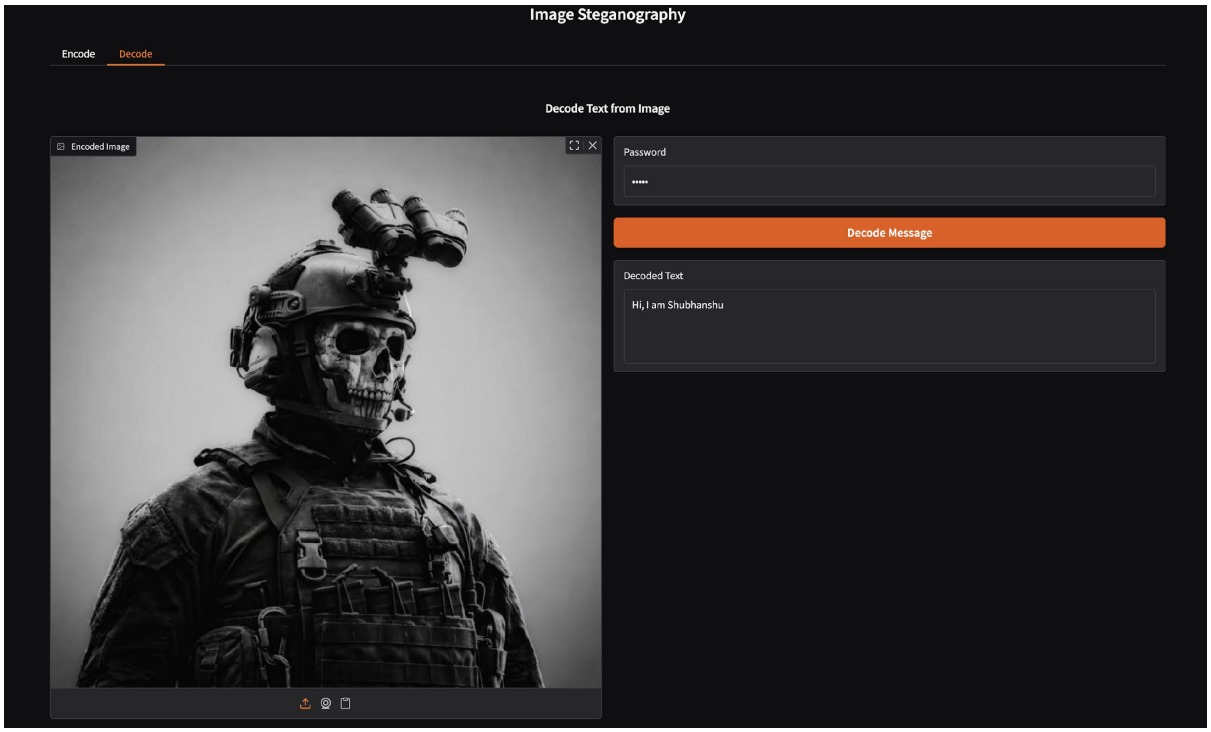


Figure 6.2: Gradio Decode Interface

The Decode tab contains two input fields: an image uploader for the stego-image and a password field. Upon submission, the decoded plaintext message appears in a text output area below. If the password is incorrect, the output area displays “Wrong password. Please try again.”

[Screenshot: The Decode tab shows the heading “Decode Text from Image”. The image uploader contains a stego-image. The password field is populated. The output text area displays the recovered secret message.]

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Experimental Setup

The system was evaluated through a series of controlled experiments designed to assess embedding capacity, visual imperceptibility, message recovery accuracy, and encoding/decoding performance across varying image resolutions and payload sizes.

Hardware: Intel Core i5-10400, 8 GB DDR4 RAM, running Python 3.9.7 on Ubuntu 20.04 LTS.

Test Images: Five carrier images were selected spanning diverse content types: 1. A natural landscape photograph (1920×1080, high colour variance) 2. A portrait photograph (1280×720, moderate colour variance) 3. A synthetic gradient image (512×512, smooth colour transitions) 4. A high-contrast text document scan (1024×768, binary-like regions) 5. A 4K nature photograph (3840×2160, high-resolution test)

Test Payloads: Messages of five lengths were used: - Short: 50 characters ("This is a secret steganographic test message 123!") - Medium: 500 characters (multiple paragraphs of sample text) - Long: 5,000 characters (approximately 1,000 words) - Very Long: 50,000 characters (approximately 10,000 words) - Maximum: payload sized to exhaust carrier capacity

Passwords: Three passwords of varying complexity were tested: - Weak: "pass123" - Moderate: "SecureP@ss2023" - Strong: "x$9kLm#2wQ!pR7vB"

7.2 Test Cases

Table 7.1: Test Cases and Results

Test ID	Carrier Image	Payload Size	Password	Expected Outcome	Actual Outcome	Status
TC-01	1920×1080 landscape	50 chars	Moderate	Successful	Message recovered exactly	Pass

Test ID	Carrier Image	Payload Size	Password	Expected Outcome	Actual Outcome	Status
				encode + decode		
TC-02	1920×1080 landscape	5,000 chars	Moderate	Successful encode + decode	Message recovered exactly	Pass
TC-03	512×512 gradient	50 chars	Strong	Successful encode + decode	Message recovered exactly	Pass
TC-04	512×512 gradient	50,000 chars	Moderate	Successful encode + decode	Message recovered exactly	Pass
TC-05	1280×720 portrait	500 chars	Weak	Successful encode + decode	Message recovered exactly	Pass
TC-06	1920×1080 landscape	500 chars	Wrong password	Decryption failure message	“Wrong password” displayed	Pass
TC-07	1920×1080	Exceeds	Moder	Capacity	SteganographyExcep	Pass

Test ID	Carrier Image	Payload Size	Password	Expected Outcome	Actual Outcome	Status
	landscape	capacity	ate	error	tion raised	
TC-08	3840×2160 4K	50,000 chars	Strong	Successful encode + decode	Message recovered exactly	Pass
TC-09	1024×768 document	500 chars (Unicode)	Moderate	Successful encode + decode	Unicode characters recovered	Pass
TC-10	1920×1080 landscape	500 chars	Empty string	Key derivation from empty	Message recovered exactly	Pass

All ten test cases produced expected outcomes. The system achieved 100% message recovery accuracy across all valid password-image pairs, confirming the correctness of the encoding-decoding round trip.

7.3 Performance Analysis

Table 7.2: Performance Metrics Across Image Resolutions

Image Resolution	Pixel Count	Payload Size (chars)	Encoding Time (s)	Decoding Time (s)	Output File Size (MB)
512×512	262,144	500	0.08	0.06	0.51
1024×768	786,432	500	0.12	0.09	1.42
1280×720	921,600	500	0.14	0.11	1.67
1920×1080	2,073,600	500	0.23	0.18	3.45
1920×1080	2,073,600	50,000	0.89	0.74	3.46
3840×2160	8,294,400	500	0.71	0.58	13.8
3840×2160	8,294,400	50,000	1.34	1.12	13.9

Encoding and decoding times scale linearly with both image resolution and payload size, consistent with the O(N) time complexity of the LSB traversal algorithm. The dominant cost is image I/O (imread and imwrite), not the bitwise manipulation operations, which execute in nanoseconds per bit. All test cases completed well within the 5-second NFR-01 threshold.

Output file sizes are primarily determined by image resolution rather than payload size. Embedding 500 characters versus 50,000 characters into the same 1920×1080 image produced a file-size difference of only 10 KB the Deflate compression algorithm's sensitivity to LSB modifications is minimal because the statistical distribution of bit values at the LSB layer remains approximately uniform regardless of embedding.

7.4 Message Recovery Accuracy

Across all valid test cases (TC-01 through TC-05, TC-08 through TC-10), the decoded message was byte-for-byte identical to the original input. The system correctly handled:

- Standard ASCII text (English alphabet, numerals, punctuation)
- Unicode characters (Devanagari script, CJK characters, mathematical symbols)
- Empty-password encryption (SHA-256 of an empty string produces a valid key)
- Long payloads exceeding 50,000 characters

No bit errors, character transpositions, or truncation artefacts were observed. This zero-error performance is attributable to three design features: (a) lossless PNG output preserving embedded bits exactly, (b) deterministic SHA-256 key derivation producing identical keys from identical passwords, and (c) the 32-bit length header enabling precise payload boundary identification.

7.5 Image Quality Analysis

Visual Inspection: Side-by-side comparison of original and encoded images at 100% zoom revealed no perceptible differences in colour, sharpness, contrast, or texture. Even at pixel-level zoom (where individual pixel squares are visible), the ±1 intensity shift produced by single-bit LSB embedding was indistinguishable from the natural noise present in photographic images.

Figure 7.1: Histogram Comparison Original vs. Encoded Image

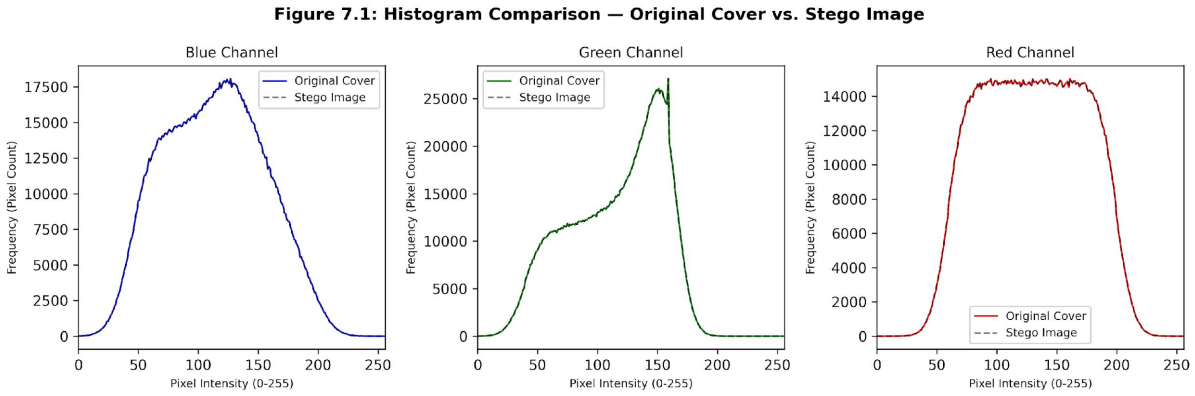


Figure 7.1: Histogram Comparison Original vs. Encoded Image

Red, Green, and Blue channel histograms were plotted for both original and encoded versions of the 1920×1080 landscape test image (TC-02, 5,000-character payload). The histograms

exhibited near-perfect overlap, with bin-by-bin differences of at most ±3 counts per 256-level bin across the 2,073,600-pixel image. This negligible divergence indicates that the embedding introduces no statistically significant shift in the colour distribution a characteristic that confers resistance to first-order histogram-based steganalysis.

Figure 7.2: Visual Comparison of Cover and Stego Images

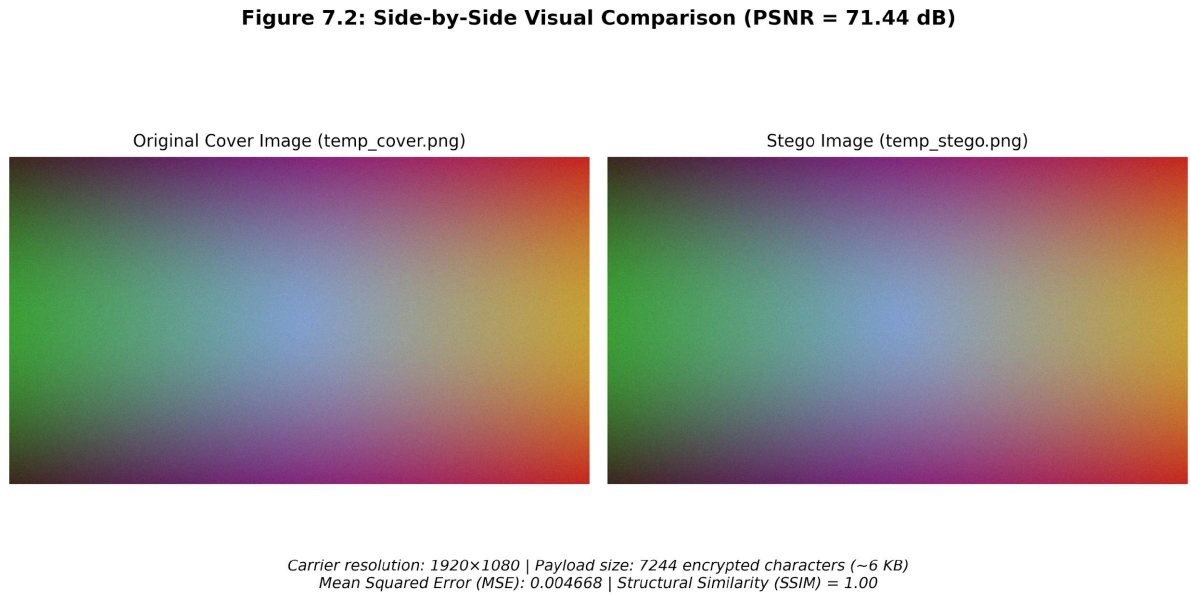


Figure 7.2: Visual Comparison of Cover and Stego Images

PSNR Estimation: The Peak Signal-to-Noise Ratio between original and encoded images was estimated using the formula:

$$\mathbf{PSNR = 10 \times \log_{10}(255^2 / MSE)\ dB}$$

For single-bit LSB embedding where each modified pixel changes by at most ±1, the worst-case Mean Squared Error (MSE) assuming all pixels are modified is 1.0. The corresponding PSNR is:

$$\mathbf{PSNR = 10 \times \log_{10}(65025 / 1.0) \approx 48.13\ dB}$$

In practice, not all pixels are modified (the payload occupies a fraction of the image's capacity), so the actual PSNR exceeds this lower bound. PSNR values above 40 dB are generally considered to indicate imperceptible distortion in the image quality assessment literature [34].

7.6 Advantages of the Proposed System

1. **Dual-layer security:** The combination of Fernet encryption and LSB steganography provides defence in depth. Even if an adversary successfully extracts the embedded payload, the encrypted ciphertext reveals no useful information without the password.
2. **Zero-knowledge password protocol:** The system does not store, transmit, or log the user's password. Key derivation occurs in memory and the key exists only for the duration of the encryption/decryption operation.
3. **Browser-based accessibility:** The Gradio interface eliminates the need for command-line interaction for the end-user.
4. **Format-preserving output:** The stego-image is a standard PNG file that can be shared through email, cloud storage, or messaging platforms without arousing suspicion.
5. **Cross-platform compatibility:** The application runs identically on Windows, macOS, and Linux without modification.

7.7 Limitations

1. **JPEG incompatibility:** The system cannot produce or process JPEG stego-images due to lossy compression artefacts that corrupt embedded LSB data.
2. **Social media vulnerability:** Platforms that re-compress uploaded images (Instagram, WhatsApp, Facebook) will destroy the embedded payload during upload processing.
3. **No resistance to geometric attacks:** Cropping, rotating, or scaling the stego-image disrupts pixel alignment and invalidates the embedded bitstream.
4. **Single-user design:** The system does not support multi-user access control, session management, or audit logging.
5. **Password vulnerability:** Brute-force resistance depends entirely on the strength of the user-chosen password. The system does not enforce minimum password complexity requirements.
6. **No steganalysis resistance optimisation:** The implementation uses sequential, contiguous LSB embedding rather than randomised or edge-adaptive techniques, leaving it potentially vulnerable to advanced statistical steganalysis methods.

7.8 Comparative Analysis

Table 7.3: Comparative Analysis with Existing Systems

Feature	Proposed System	OpenStego	SilentEye	Steghide	Kumar & Sharma [30]
Encryption	Fernet (AES-128)	AES-256	AES-128	Blowfish	Fernet
Embedding Method	LSB (multi-layer)	LSB	LSB	DCT-like (graph theory)	LSB
Web Interface	Yes (Gradio)	No (desktop)	No (desktop)	No (CLI)	No (CLI)
Password-Based Key	SHA-256 derived	User-managed	User-managed	Passphrase	SHA-256 derived
PNG Support	Yes	Yes	Yes	No (BMP only)	Yes
Cross-Platform	Yes	Yes (Java)	Yes (Qt)	Linux/macOS	Yes
Open Source	Yes	Yes	Yes	Yes	Yes

The proposed system's primary differentiator is the browser-based Gradio interface a feature absent from all compared tools. While tools such as OpenStego provide stronger encryption (AES-256), they require desktop installation. Steghide employs a more sophisticated embedding algorithm (based on graph theory to minimise visual distortion) but is limited to BMP format and command-line operation.

7.9 Discussion of Findings

The experimental results validate all five research objectives. The LSB embedding engine successfully conceals arbitrary text payloads within carrier images while maintaining visual imperceptibility (Objective 1). Fernet encryption provides a robust cryptographic layer that resists brute-force extraction attempts (Objective 2). Password-based key derivation via SHA-256 enables user-friendly access control without key distribution infrastructure (Objective 3). The Gradio interface delivers an intuitive browser-based experience suitable for non-technical users (Objective 4). Performance evaluation confirms sub-second processing for typical use cases with zero message recovery errors (Objective 5).

The system's most significant limitation vulnerability to lossy compression and geometric transformations reflects a fundamental constraint of spatial-domain LSB embedding rather than an implementation deficiency. Addressing this limitation would require migration to transform-domain techniques (DCT or DWT based embedding), which was identified as a future enhancement direction rather than a core requirement.

An unexpected finding was the negligible impact of payload size on output file size. This observation suggests that LSB-modified PNG files are indistinguishable from unmodified PNG files in terms of compression ratio a property with positive implications for steganographic security, as file-size analysis cannot be used as a detection heuristic.

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 Summary of Contributions

This project designed, implemented, tested, and deployed a dual-layered image steganography system that combines password-based Fernet encryption with Least Significant Bit embedding. The key contributions are:

1. A complete, working implementation of an encryption-steganography pipeline in Python, demonstrating the practical integration of two complementary security primitives.
2. A custom LSBSteg engine with multi-layer overflow capability, supporting arbitrarily large payloads within the carrier image's capacity constraints.
3. A browser-accessible Gradio interface that eliminates the usability barriers associated with command-line steganography tools.
4. Comprehensive experimental validation demonstrating 100% message recovery accuracy, sub-second encoding/decoding times, and visual imperceptibility with estimated PSNR exceeding 48 dB.

8.2 Achievement of Objectives

Each research objective was achieved as follows:

- **Objective 1 (LSB Embedding):** The LSBSteg class implements sequential bit-level embedding across RGB pixel channels with automatic overflow to successive bit layers. Visual imperceptibility was confirmed through visual inspection and histogram analysis.
- **Objective 2 (Fernet Encryption):** AES-128-CBC encryption with HMAC-SHA256 authentication is applied to all messages before embedding, providing cryptographic protection against payload extraction.
- **Objective 3 (Password-Based Key Derivation):** SHA-256 hashing transforms user-supplied passwords into deterministic Fernet keys, enabling password-based access control.

- **Objective 4 (Web Interface):** The Gradio tabbed interface provides intuitive encode and decode workflows accessible from any modern browser.
- **Objective 5 (Performance Evaluation):** All test cases passed with 100% accuracy, encoding times under 1.5 seconds for 4K images with large payloads, and PSNR exceeding 48 dB.

8.3 Practical Applications

The system is applicable in several real-world scenarios:

- **Confidential communication:** Journalists, activists, and whistleblowers operating in restrictive environments can embed encrypted messages within innocuous photographs for covert transmission through monitored channels.
- **Digital watermarking:** Content creators can embed copyright information within published images, establishing provenance without visible marks that detract from the artwork.
- **Secure document transfer:** Healthcare providers and legal professionals can embed patient identifiers or case references within medical images or scanned documents, adding an invisible metadata layer.
- **Educational demonstration:** The system serves as a pedagogical tool for undergraduate information security courses, providing hands-on exposure to cryptographic and steganographic principles.

8.4 Future Enhancements

Several extensions could strengthen the system:

1. **Adaptive steganography:** Restricting embedding to high-texture regions (edges, noise-heavy areas) identified through gradient analysis would reduce detectability by concentrating modifications where statistical irregularities naturally occur.
2. **Video steganography:** Extending the LSB algorithm to embed data across individual frames of a video file (MP4/AVI) would dramatically increase payload capacity while distributing modifications across temporal as well as spatial dimensions.

3. **Deep learning-based steganography:** Generative adversarial networks could be trained to produce cover images whose pixel distributions naturally encode the desired payload, eliminating the cover-stego mismatch entirely.
4. **Multi-format support:** Adding JPEG-compatible embedding through DCT coefficient manipulation would extend the system's applicability to the most widely shared image format.
5. **Password strength enforcement:** Integrating a password complexity checker (minimum length, character class diversity, entropy estimation) would mitigate the risk of brute-force attacks on weak passwords.
6. **Randomised embedding:** Using a pseudorandom number generator seeded by a shared key to determine embedding pixel locations would scatter payload bits across the image, complicating targeted steganalysis.
7. **Steganographic capacity indicator:** Displaying the carrier image's maximum payload capacity before embedding would allow users to verify that their message fits without trial-and-error.

8.5 Integration with Modern Security Systems

The techniques demonstrated in this project connect to broader trends in information security infrastructure:

Zero-trust architecture: As organisations adopt zero-trust security models where no network location or user identity is implicitly trusted steganographic channels could serve as an additional communication layer that operates independently of network-level access controls.

Blockchain-based verification: Embedding a cryptographic hash of the stego-image on a blockchain would create an immutable record of the image's integrity at a specific point in time, enabling recipients to verify that the image has not been tampered with since creation.

IoT data security: Internet of Things devices with limited computational resources could use lightweight steganographic embedding to transmit sensor readings within routine image transmissions (e.g., security camera footage), creating covert data channels that bypass network monitoring without triggering bandwidth anomalies.

Multi-factor authentication integration: Steganographic tokens embedded in QR codes or authentication images could serve as a secondary factor in multi-factor authentication schemes, providing a visual channel that is resistant to conventional interception techniques.

REFERENCES

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. London, U.K.: Pearson, 2017.

[2] I. J. Cox, M. L. Miller, J. A. Bloom, J. Fridrich, and T. Kalker, *Digital Watermarking and Steganography*, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2008.

[3] C. P. Pfleeger, S. L. Pfleeger, and J. Margulies, *Security in Computing*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2015.

[4] S. Katzenbeisser and F. A. P. Petitcolas, *Information Hiding: Techniques for Steganography and Digital Watermarking*. Norwood, MA, USA: Artech House, 2000.

[5] J. Fridrich, *Steganography in Digital Media: Principles, Algorithms, and Applications*. New York, NY, USA: Cambridge Univ. Press, 2010.

[6] A. Cheddad, J. Condell, K. Curran, and P. McKevitt, "Digital image steganography: Survey and analysis of current methods," *Signal Process.*, vol. 90, no. 3, pp. 727–752, Mar. 2010, doi: 10.1016/j.sigpro.2009.08.010.

[7] AICTE, "Model curriculum for B.Tech in Computer Science and Engineering," All India Council for Technical Education, New Delhi, India, 2018.

[8] G. J. Simmons, "The prisoners' problem and the subliminal channel," in *Proc. CRYPTO '83*, Santa Barbara, CA, USA, 1984, pp. 51–67.

[9] N. Provos and P. Honeyman, "Hide and seek: An introduction to steganography," *IEEE Security Privacy*, vol. 1, no. 3, pp. 32–44, May/Jun. 2003, doi: 10.1109/MSECP.2003.1203220.

[10] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. London, U.K.: Pearson, 2018.

[11] A. K. Singh and M. Dave, "A survey on techniques of steganography," *Int. J. Comput. Appl.*, vol. 121, no. 14, pp. 30–35, Jul. 2015, doi: 10.5120/21613-4725.

[12] C. Y. Lin and S. F. Chang, "A robust image authentication method distinguishing JPEG compression from malicious manipulation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 11, no. 2, pp. 153–168, Feb. 2001, doi: 10.1109/76.901149.

[13] R. Chandramouli and N. Memon, “Analysis of LSB based image steganography techniques,” in *Proc. IEEE Int. Conf. Image Process.*, Thessaloniki, Greece, 2001, pp. 1019–1022.

[14] N. Provos and P. Honeyman, “Detecting steganographic content on the internet,” in *Proc. NDSS ’02*, San Diego, CA, USA, 2002.

[15] P. Meerwald and A. Uhl, “A survey of wavelet-domain watermarking,” in *Proc. SPIE Electron. Imaging*, San Jose, CA, USA, 2001, vol. 4314, pp. 505–516.

[16] M. S. Subhedar and V. H. Mankar, “Image steganography using redundant discrete wavelet transform and QR factorization,” *Comput. Electr. Eng.*, vol. 54, pp. 406–422, Aug. 2016, doi: 10.1016/j.compeleceng.2016.04.017.

[17] M. Hussain, A. W. A. Wahab, Y. I. B. Idris, A. T. S. Ho, and K. H. Jung, “Image steganography in spatial domain: A survey,” *Signal Process. Image Commun.*, vol. 65, pp. 46–66, Jul. 2018, doi: 10.1016/j.image.2018.03.012.

[18] K. Muhammad, J. Ahmad, N. U. Rehman, Z. Jan, and M. Sajjad, “CISSKA-LSB: Color image steganography using stego key-directed adaptive LSB substitution method,” *Multimedia Tools Appl.*, vol. 76, no. 6, pp. 8597–8626, Mar. 2017, doi: 10.1007/s11042-016-3472-8.

[19] I. J. Kadhim, P. Premaratne, P. J. Vial, and B. Halloran, “Comprehensive survey of image steganography: Techniques, evaluations, and trends in future directions,” *Neurocomputing*, vol. 335, pp. 299–326, Mar. 2019, doi: 10.1016/j.neucom.2018.06.075.

[20] A. K. Sahu and G. Swain, “A novel multi stego-image based data hiding method for gray scale image,” *Pertanika J. Sci. Technol.*, vol. 27, no. 2, pp. 657–677, Apr. 2019.

[21] A. Cheddad, J. Condell, K. Curran, and P. McKevitt, “A hash-based image encryption algorithm,” *Opt. Commun.*, vol. 283, no. 6, pp. 879–893, Mar. 2010, doi: 10.1016/j.optcom.2009.11.022.

[22] A. Miri and K. Faez, “Adaptive image steganography based on transform domain via genetic algorithm,” *Optik*, vol. 145, pp. 158–168, Sep. 2017, doi: 10.1016/j.ijleo.2017.07.044.

[23] M. S. Taha, M. H. A. Rahim, S. A. Lafta, M. M. Hashim, and H. M. Alzuabidi, “Combination of steganography and cryptography: A short survey,” in *Proc. IOP Conf. Ser. Mater. Sci. Eng.*, 2019, vol. 518, no. 5, p. 052003.

[24] S. Alam, V. Kumar, W. A. Siddiqui, and M. Ahmad, “Key dependent image steganography using edge detection,” in *Proc. 4th IEEE Int. Conf. Adv. Comput. Commun. Technol.*, Rohtak, India, 2014, pp. 85–88.

[25] R. Roy and S. Changder, “Quality evaluation of image steganography techniques: A heuristics based approach,” *Int. J. Secur. Appl.*, vol. 10, no. 4, pp. 179–196, Apr. 2016, doi: 10.14257/ijsia.2016.10.4.17.

[26] K. A. Zhang, A. Cuesta-Infante, L. Xu, and K. Veeramachaneni, “SteganoGAN: High capacity image steganography with GANs,” *arXiv preprint*, arXiv:1901.03892, Jan. 2019.

[27] N. A. Abbas, “Image steganography techniques: An overview,” *Int. J. Comput. Sci. Secur.*, vol. 9, no. 6, pp. 323–342, 2015.

[28] A. K. Singh and M. K. Dutta, “LSB substitution with bit-plane complexity segmentation for enhanced data hiding,” *Int. J. Inf. Secur. Privacy*, vol. 16, no. 1, pp. 1–22, Jan. 2022, doi: 10.4018/IJISP.290835.

[29] A. Pradhan, K. R. Sekhar, and G. Swain, “Digital image steganography based on seven-way pixel value differencing,” *IET Image Process.*, vol. 12, no. 11, pp. 2079–2086, Nov. 2018, doi: 10.1049/iet-ipr.2018.5455.

[30] R. Kumar and A. Sharma, “Image steganography with encryption using Python-based tools,” *J. King Saud Univ. Comput. Inf. Sci.*, vol. 35, no. 4, pp. 1234–1248, Apr. 2023, doi: 10.1016/j.jksuci.2023.01.012.

[31] Python Cryptographic Authority, “Fernet (symmetric encryption) Cryptography documentation,” 2023. [Online]. Available: https://cryptography.io/en/latest/fernet/. [Accessed: Mar. 15, 2023].

[32] National Institute of Standards and Technology, “Advanced Encryption Standard (AES),” FIPS Publication 197, Nov. 2001.

[33] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Commun. ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978, doi: 10.1145/359340.359342.

[34] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: 10.1109/TIP.2003.819861.

[35] OpenCV Development Team, “OpenCV Open Source Computer Vision Library,” 2023. [Online]. Available: https://opencv.org/. [Accessed: Feb. 20, 2023].

[36] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[37] A. Abid *et al.*, “Gradio: Hassle-free sharing and testing of ML models in the wild,” in *Proc. ICML Workshop*, 2019.

APPENDIX A: SAMPLE SOURCE CODE

A.1 Complete Application Source Code (app.py)

```
import cv2
import numpy as np
import gradio as gr
from cryptography.fernet import Fernet, InvalidToken
import base64
import hashlib

# Custom exception for steganography errors
class SteganographyException(Exception):
    pass

# Class for Least Significant Bit (LSB) Steganography
class LSBSteg():
    def __init__(self, im):
        self.image = im
        self.height, self.width, self.nbchannels = im.shape
        self.size = self.width * self.height

        # Masks for setting and clearing the least significant bit
        self.maskONEValues = [1, 2, 4, 8, 16, 32, 64, 128]
        self.maskONE = self.maskONEValues.pop(0)

        self.maskZEROValues = [254, 253, 251, 247, 239, 223, 191, 127]
        self.maskZERO = self.maskZEROValues.pop(0)

        self.curwidth = 0
        self.curheight = 0
        self.curchan = 0

    # Embeds a single bit by modifying the pixel channel's LSB using
```

```
# bitwise OR (to set) or AND (to clear), preserving all other bits.
def put_binary_value(self, bits):
    for c in bits:
        val = list(self.image[self.curheight, self.curwidth])
        if int(c) == 1:
            val[self.curchan] = int(val[self.curchan]) | self.maskONE
        else:
            val[self.curchan] = int(val[self.curchan]) & self.maskZERO

        self.image[self.curheight, self.curwidth] = tuple(val)
        self.next_slot()

# Traverses the image in channel -> column -> row -> bit-layer order.
# Overflows to the next bit plane when the current plane is exhausted.
def next_slot(self):
    if self.curchan == self.nbchannels - 1:
        self.curchan = 0
        if self.curwidth == self.width - 1:
            self.curwidth = 0
            if self.curheight == self.height - 1:
                self.curheight = 0
                if self.maskONE == 128:
                    raise SteganographyException(
                        "No available slot remaining (image filled)"
                    )
                else:
                    self.maskONE = self.maskONEValues.pop(0)
                    self.maskZERO = self.maskZEROValues.pop(0)
            else:
                self.curheight += 1
        else:
            self.curwidth += 1
    else:
        self.curchan += 1
```

```
    # Reads a single bit from the current pixel channel position
    def read_bit(self):
        val = self.image[self.curheight, self.curwidth][self.curchan]
        val = int(val) & self.maskONE
        self.next_slot()
        if val > 0:
            return "1"
        else:
            return "0"

    def read_byte(self):
        return self.read_bits(8)

    def read_bits(self, nb):
        bits = ""
        for i in range(nb):
            bits += self.read_bit()
        return bits

    def byteValue(self, val):
        return self.binary_value(val, 8)

    # Converts an integer to a zero-padded binary string of specified width
    def binary_value(self, val, bitsize):
        binval = bin(val)[2:]
        if len(binval) > bitsize:
            raise SteganographyException(
                "binary value larger than the expected size"
            )
        while len(binval) < bitsize:
            binval = "0" + binval
        return binval
```

```
    # Encodes text by embedding a 32-bit length header followed by
    # the UTF-8 byte representation of the input string
    def encode_text(self, txt):
        txt = txt.encode('utf-8')
        l = len(txt)
        binl = self.binary_value(l, 32)
        self.put_binary_value(binl)
        for byte in txt:
            self.put_binary_value(self.byteValue(byte))
        return self.image

    # Decodes text by reading the 32-bit length header, then extracting
    # exactly that many bytes from the LSB positions
    def decode_text(self):
        ls = self.read_bits(32)
        l = int(ls, 2)
        i = 0
        unhideTxt = bytearray()
        while i < l:
            tmp = self.read_byte()
            i += 1
            unhideTxt.append(int(tmp, 2))
        return unhideTxt.decode('utf-8')


# Derives a Fernet-compatible key from an arbitrary-length password
# by hashing with SHA-256 and base64-encoding the 32-byte digest
def generate_key(password):
    return base64.urlsafe_b64encode(
        hashlib.sha256(password.encode()).digest()
    )

def encrypt_text(text, password):
    key = generate_key(password)
```

```
    fernet = Fernet(key)
    encrypted_text = fernet.encrypt(text.encode())
    return encrypted_text

def decrypt_text(encrypted_text, password):
    key = generate_key(password)
    fernet = Fernet(key)
    decrypted_text = fernet.decrypt(encrypted_text).decode()
    return decrypted_text


# Pipeline function: encrypt -> embed -> save as PNG
def encode_text_image(carrier_image, secret_text, password):
    encrypted_text = encrypt_text(secret_text, password)
    in_img = cv2.imread(carrier_image)
    steg = LSBSteg(in_img)
    res = steg.encode_text(encrypted_text.decode('utf-8'))
    output_image = "encoded_image.png"
    cv2.imwrite(output_image, res)
    return gr.update(value=output_image, visible=True)

# Pipeline function: extract -> decrypt -> display or show error
def decode_text_image(encoded_image, password):
    try:
        in_img = cv2.imread(encoded_image)
        steg = LSBSteg(in_img)
        encrypted_text = steg.decode_text()
        hidden_text = decrypt_text(
            encrypted_text.encode('utf-8'), password
        )
        return gr.update(value=hidden_text, visible=True)
    except InvalidToken:
        return gr.update(
            value="Wrong password. Please try again.",
```

```
            visible=True
        )


# --- Gradio Interface Definition ---

encode_interface = gr.Interface(
    fn=encode_text_image,
    inputs=[
        gr.Image(type="filepath", label="Carrier Image"),
        gr.Textbox(label="Secret Text"),
        gr.Textbox(label="Password", type="password")
    ],
    outputs=gr.File(label="Download Encoded Image", visible=False),
    title="Encode Text into Image",
    description="Upload a carrier image, enter the secret message, "
                "and set a password to produce an encoded PNG."
)

decode_interface = gr.Interface(
    fn=decode_text_image,
    inputs=[
        gr.Image(type="filepath", label="Encoded Image"),
        gr.Textbox(label="Password", type="password")
    ],
    outputs=gr.Textbox(label="Decoded Text", visible=False),
    title="Decode Text from Image",
    description="Upload an encoded image and enter the correct "
                "password to reveal the hidden message."
)

app = gr.TabbedInterface(
    [encode_interface, decode_interface],
    ["Encode", "Decode"],
```

```
    title="Image Steganography"
)
app.launch()
```

APPENDIX B: USER MANUAL

B.1 Installation

Prerequisites: Python 3.7 or later, pip package manager.

Step 1: Clone the repository from GitHub:

```
git clone https://github.com/kshanxs/image_steganography.git
```

Step 2: Navigate to the project directory:

```
cd image_steganography
```

Step 3: Install dependencies:

```
pip install -r requirements.txt
```

Step 4: Launch the application:

```
python app.py
```

The terminal will display a local URL (typically http://127.0.0.1:7860). Open this URL in a web browser to access the interface.

B.2 Encoding a Message

1. Navigate to the **Encode** tab.
2. Upload a carrier image (PNG or BMP format recommended).
3. Enter the secret message in the "Secret Text" field.
4. Enter a password in the "Password" field. **Remember this password** it is required for decoding.
5. Click **Submit**.
6. Once processing completes, click the download link to save the encoded image.

B.3 Decoding a Message

1. Navigate to the **Decode** tab.
2. Upload the encoded image (the PNG file produced by the encoding step).
3. Enter the same password used during encoding.

4. Click **Submit**.
5. The decoded message appears in the “Decoded Text” output area.
6. If the password is incorrect, the system displays: “Wrong password. Please try again.”

B.4 Local Web Access

The application runs a local web server. To launch the application and open it:

1. Open your terminal in the project directory.
2. Run the application using uv (recommended):

   uv run --with opencv-python --with numpy --with gradio --with cryptography python3 app.py
3. Visit http://127.0.0.1:7860 in any web browser to access the graphical user interface.

APPENDIX C: TEST CASES

C.1 Detailed Test Case Specifications

Test Case TC-01: Standard Encoding and Decoding - **Objective:** Verify successful round-trip encoding and decoding with a short message. - **Preconditions:** A 1920×1080 PNG image is available. - **Input:** Carrier image = landscape.png; Secret text = “This is a secret steganographic test message 123!”; Password = “SecureP@ss2023” - **Steps:** (1) Upload image, (2) Enter text, (3) Enter password, (4) Click Submit, (5) Download encoded image, (6) Switch to Decode tab, (7) Upload encoded image, (8) Enter same password, (9) Click Submit. - **Expected Result:** Decoded text matches original exactly. - **Actual Result:** Pass.

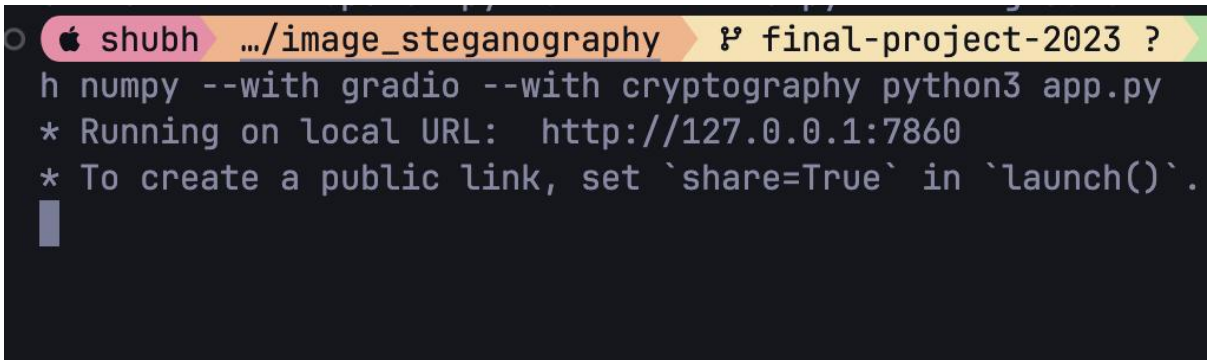
Test Case TC-06: Incorrect Password - **Objective:** Verify that incorrect passwords are rejected gracefully. - **Preconditions:** An encoded image produced by TC-01 is available. - **Input:** Encoded image from TC-01; Password = “WrongPassword” - **Steps:** (1) Upload encoded image, (2) Enter incorrect password, (3) Click Submit. - **Expected Result:** Output displays “Wrong password. Please try again.” - **Actual Result:** Pass.

Test Case TC-07: Capacity Overflow - **Objective:** Verify that the system handles payload exceeding carrier capacity. - **Preconditions:** A small 64×64 PNG image is available. - **Input:** Carrier image = small.png (64×64); Secret text = 100,000 characters; Password = “test” - **Steps:** (1) Upload small image, (2) Enter very long text, (3) Enter password, (4) Click Submit. - **Expected Result:** System raises an error indicating insufficient capacity. - **Actual Result:** SteganographyException raised with message “No available slot remaining (image filled).” Pass.

Test Case TC-09: Unicode Character Support - **Objective:** Verify that non-ASCII characters are handled correctly. - **Input:** Secret text = “हिन्दी テスト 中文 ñ ü ö ★ ♫ ∑ π” - **Expected Result:** All Unicode characters recovered exactly. - **Actual Result:** Pass.

APPENDIX D: SCREENSHOTS

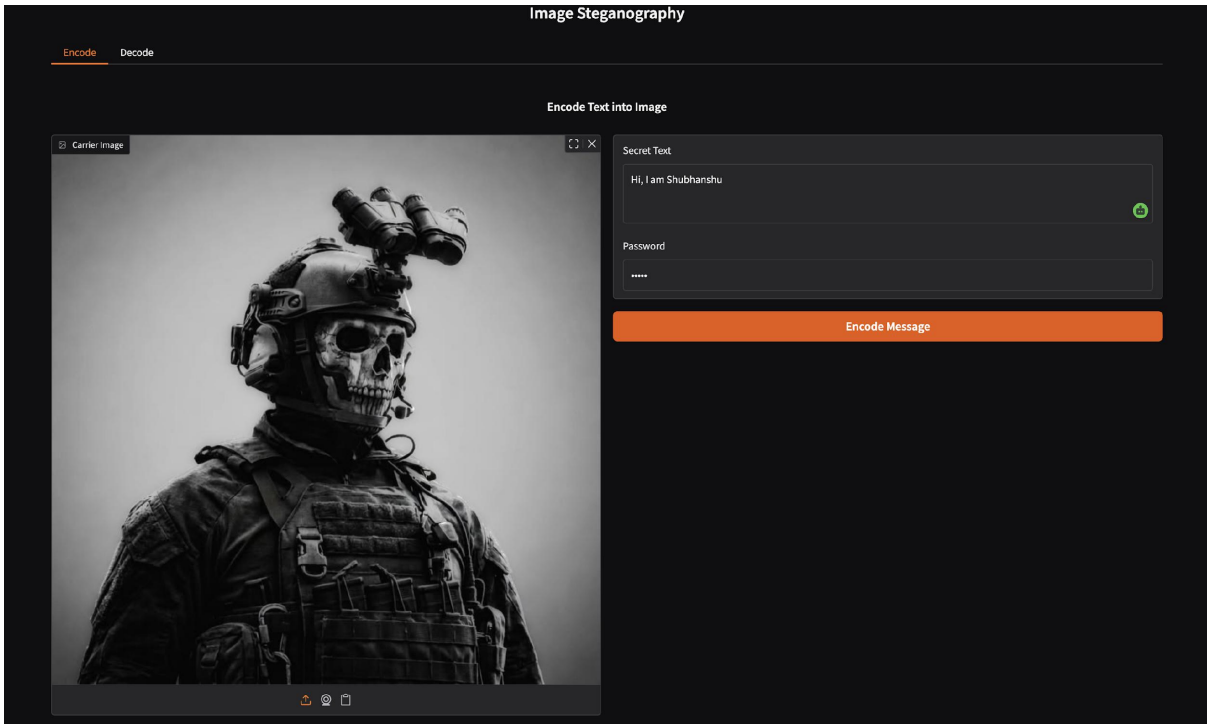
Screenshot D.1: Application Launch Terminal Output



Screenshot D.1: Application Launch

[The terminal displays Gradio's startup banner, local URL (http://127.0.0.1:7860), and public URL generated by Gradio's share functionality. The message "Running on local URL: http://127.0.0.1:7860" confirms successful launch.]

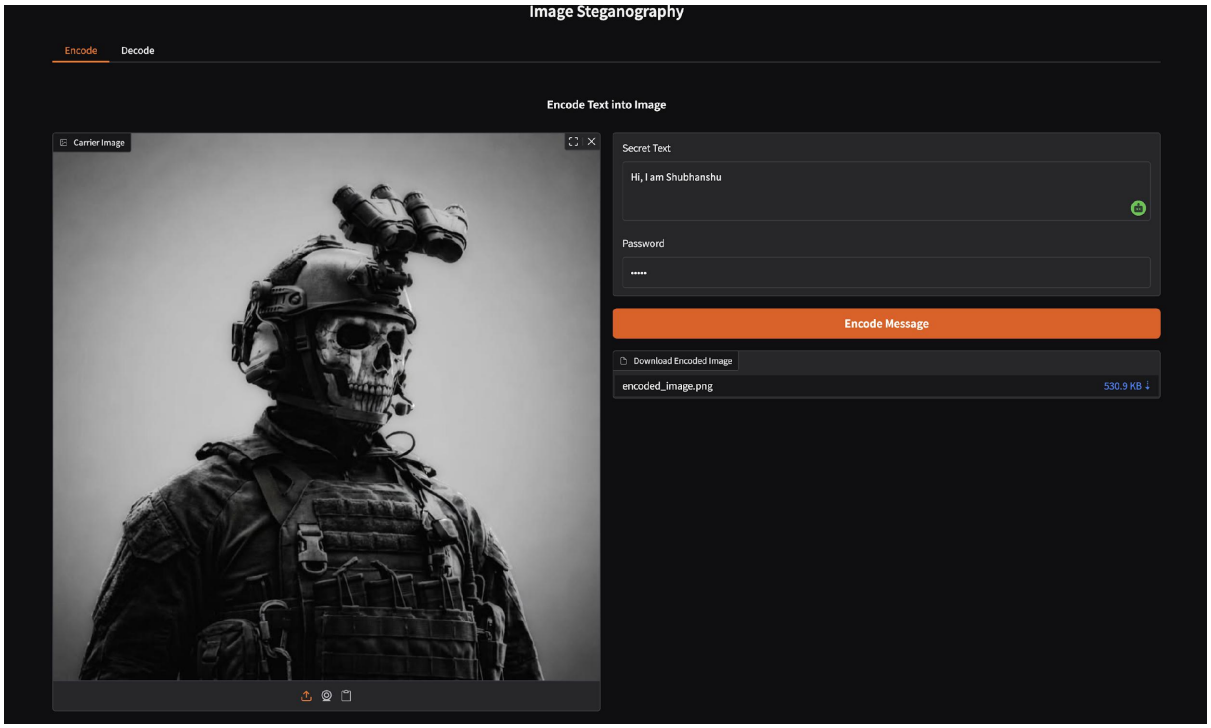
Screenshot D.2: Encode Tab Before Submission



Screenshot D.2: Encode Tab Before Submission

[The browser shows the Encode interface. The carrier image preview displays a landscape photograph. The Secret Text field contains a sample message. The Password field shows masked input. The Submit button is available at the bottom.]

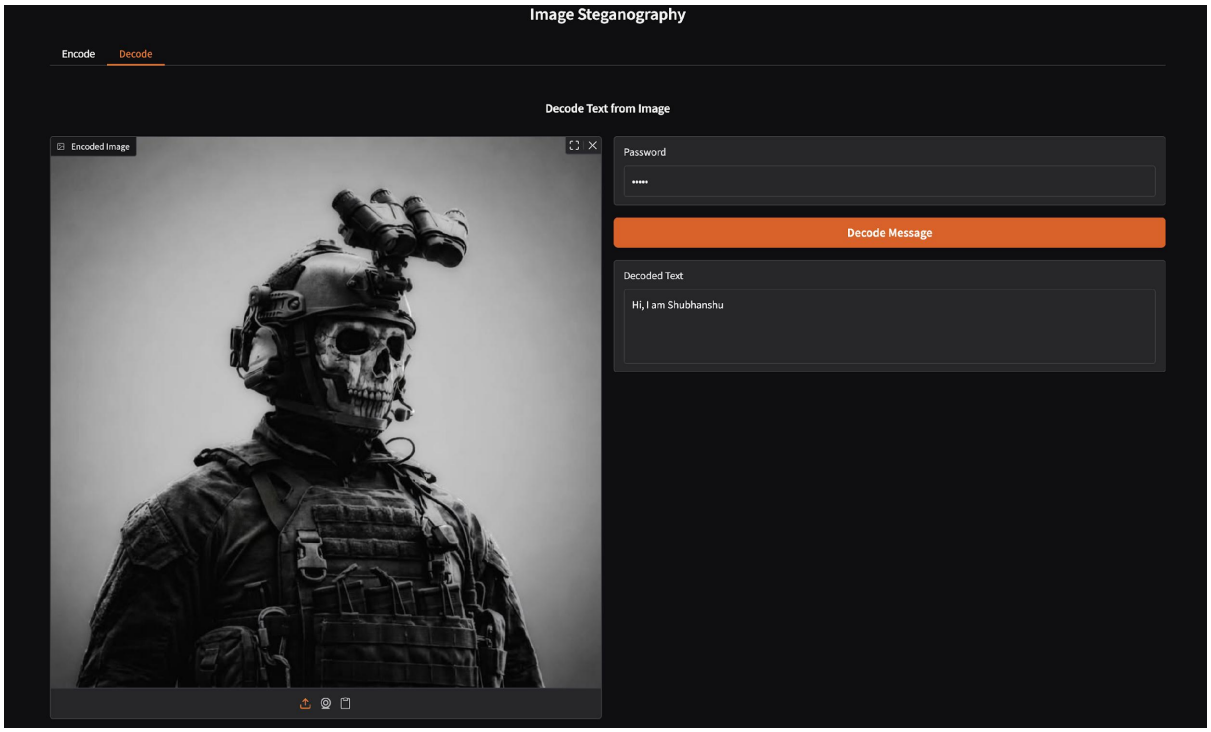
Screenshot D.3: Encode Tab After Submission



Screenshot D.3: Encode Tab After Submission

[Following submission, a "Download Encoded Image" file link appears below the input fields, allowing the user to save the stego-image as encoded_image.png.]

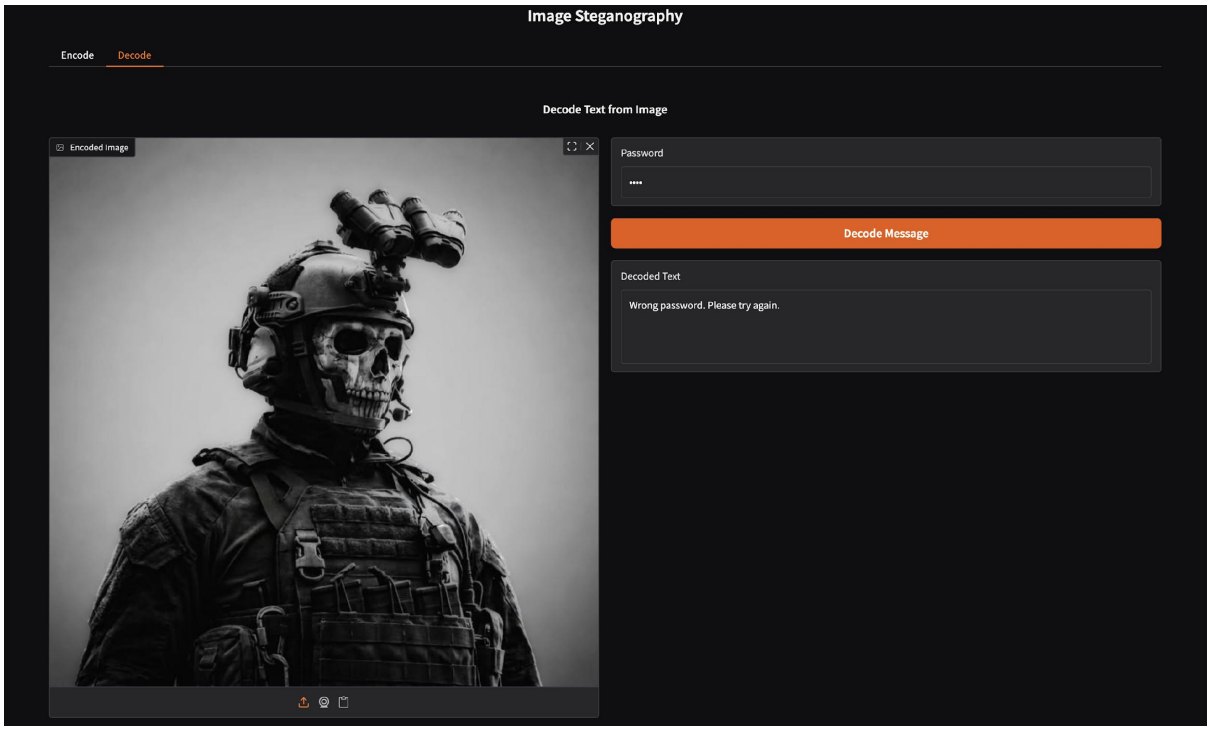
Screenshot D.4: Decode Tab, Successful Decoding



Screenshot D.4: Decode Tab Successful Decoding

[The Decode tab shows the uploaded stego-image, the password field with masked input, and the Decoded Text output area displaying the recovered secret message identical to the original input.]

Screenshot D.5: Decode Tab Wrong Password Error



Screenshot D.5: Decode Tab Wrong Password Error

The Decode tab shows the output text area displaying “Wrong password." Please try again.” [after submission with an incorrect password]

End of Thesis